DATA 260 – HW 6

7- Community Sports League Fixtures

SJSU ID : 020053319 | SRI PRAJEET VILLYAL

Hardware: MacBook Pro, Apple M2, 8 GB RAM, macOS

Local model used: qwen2.5:3b (run locally through Ollama)

GitHub repository: https://github.com/villyalsriprajeet-cmd/data260-3319

Section 0 - Personal Configuration and Domain Value Definition

Value	Definition
SID4	3319
PORT_BASE	8619
PREFIX	s3319
SEED	3319
VERIFY_SEED	263319
DOMAIN_ID	7

Part 1 – MongoDB Document API

1. Collection and document model

The fixtures collection lives in the s3319_doc database, on a MongoDB 7.0 Docker container (s3319_mongo). Each document is one fixture. The rules are enforced twice: by Pydantic in the API and by a $jsonSchema validator on the collection itself.

Field	Type	Rule
fixture_title	string	required, 1 to 100 characters
description	string or null	optional
status	enum	Scheduled, Live, Completed, Postponed; default Scheduled
category	enum	League, Cup, Friendly, Tournament; required
kickoff_at	date	required, ISO date/time
created_at, updated_at	date	set by the API; updated_at changes on every update

2. Code

The code is split into separate files: config.py (settings), mongo_db.py (PyMongo connection, collection rules, indexes), doc_schemas.py (Pydantic schemas), routers/docs.py (routes) and main.py (CORS and error handling). FastAPI reads and returns JSON on its own, and the CORS middleware allows the React dev server on port 5173.

Settings (code/api/config.py)

```
SID4 = 3319  # last four digits of the student id
PORT_BASE = 8000 + SID4 % 900  # 8619
PREFIX = f"s{SID4}"
SEED = SID4
VERIFY_SEED = 260000 + SID4
MONGODB_URI = os.getenv("MONGODB_URI", "mongodb://localhost:27017")  # the Docker container from Step 1
MONGODB_DB = f"{PREFIX}_doc"  # s3319_doc
FIXTURES_COLLECTION = "fixtures"  # Part 1 document collection
STATUS_VALUES = ["Scheduled", "Live", "Completed", "Postponed"]  # same dropdown as HW1
CATEGORY_VALUES = ["League", "Cup", "Friendly", "Tournament"]  # fixed set of competition types
```

PyMongo connection and collection rules (code/api/mongo_db.py)

```
_client = None  # one shared client for the whole app
def get_client() -> MongoClient:
    global _client
    if _client is None:
        _client = MongoClient(MONGODB_URI, serverSelectionTimeoutMS=3000)  # fail fast if Mongo is down
    return _client
def get_db():
    return get_client()[MONGODB_DB]
def get_fixtures():
# ...
    return get_db()[FIXTURES_COLLECTION]
# ...
FIXTURES_VALIDATOR = {"$jsonSchema": {
    "bsonType": "object",
    "required": ["fixture_title", "status", "category", "kickoff_at", "created_at", "updated_at"],  # mandatory fields
    "properties": {
        "fixture_title": {"bsonType": "string", "minLength": 1, "maxLength": 100},  # required short text
        "description": {"bsonType": ["string", "null"]},  # optional text
        "status": {"enum": STATUS_VALUES},  # constrained field with a default set by the API
        "category": {"enum": CATEGORY_VALUES},  # constrained field with a fixed set
        "kickoff_at": {"bsonType": "date"},  # required date/time
        "created_at": {"bsonType": "date"},
        "updated_at": {"bsonType": "date"},
    }}}
```

Collection creation and indexes (code/api/mongo_db.py)

```
def ensure_collections():
    db = get_db()
    existing = db.list_collection_names()
    for name, (validator, indexes) in COLLECTIONS.items():
        if name not in existing:
            db.create_collection(name, validator=validator)  # create with rules
        else:
            db.command("collMod", name, validator=validator)  # refresh the rules
        for keys in indexes:
            db[name].create_index(keys)  # speed up the lookups the memory code does
```

Pydantic schemas for create (code/api/doc_schemas.py)

```
StatusEnum = Literal["Scheduled", "Live", "Completed", "Postponed"]  # must match config.STATUS_VALUES
CategoryEnum = Literal["League", "Cup", "Friendly", "Tournament"]  # must match config.CATEGORY_VALUES
class FixtureDocCreate(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")  # trim spaces, reject unknown fields
    fixture_title: str = Field(min_length=1, max_length=100)  # required short text
    description: Optional[str] = None  # optional text
    status: StatusEnum = "Scheduled"  # default status
    category: CategoryEnum  # required, fixed set
    kickoff_at: datetime  # required ISO date/time
```

Pydantic schema for update (code/api/doc_schemas.py)

```
class FixtureDocUpdate(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    fixture_title: Optional[str] = Field(default=None, min_length=1, max_length=100)  # only sent fields change
    description: Optional[str] = None
    status: Optional[StatusEnum] = None
    category: Optional[CategoryEnum] = None
    kickoff_at: Optional[datetime] = None
    @model_validator(mode="after")
    def no_null_required(self):
        for name in ("fixture_title", "status", "category", "kickoff_at"):
            if name in self.model_fields_set and getattr(self, name) is None:
                raise ValueError(f"{name} cannot be null")  # required fields may be changed but not cleared
        return self
```

POST and GET all (code/api/routers/docs.py)

```
router = APIRouter(prefix="/api/fixtures", tags=["Fixtures (MongoDB)"])
def to_object_id(value: str) -> ObjectId:
    try:
        return ObjectId(value)
    except (InvalidId, TypeError):
        raise HTTPException(status_code=400, detail=f"Invalid id format: {value}")  # not a 24-char hex id
def serialize(doc: dict) -> dict:
    doc["id"] = str(doc.pop("_id"))  # ObjectId is not JSON friendly
    return doc
@router.post("", response_model=FixtureDocOut, status_code=201)
def create_fixture(payload: FixtureDocCreate):
    now = datetime.now(timezone.utc)
    doc = {**payload.model_dump(), "created_at": now, "updated_at": now}  # timestamps set by the server
    doc["_id"] = get_fixtures().insert_one(doc).inserted_id
    return serialize(doc)
@router.get("", response_model=list[FixtureDocOut])
def list_fixtures(skip: int = Query(0, ge=0), limit: int = Query(100, ge=1, le=500)):
# ...
    cursor = get_fixtures().find().sort("created_at", -1).skip(skip).limit(limit)  # newest first
    return [serialize(d) for d in cursor]
```

GET one (code/api/routers/docs.py)

```
@router.get("/{fixture_id}", response_model=FixtureDocOut)
def get_fixture(fixture_id: str):
    doc = get_fixtures().find_one({"_id": to_object_id(fixture_id)})
    if doc is None:
        raise HTTPException(status_code=404, detail=f"Fixture {fixture_id} not found")
    return serialize(doc)
```

PUT and DELETE (code/api/routers/docs.py)

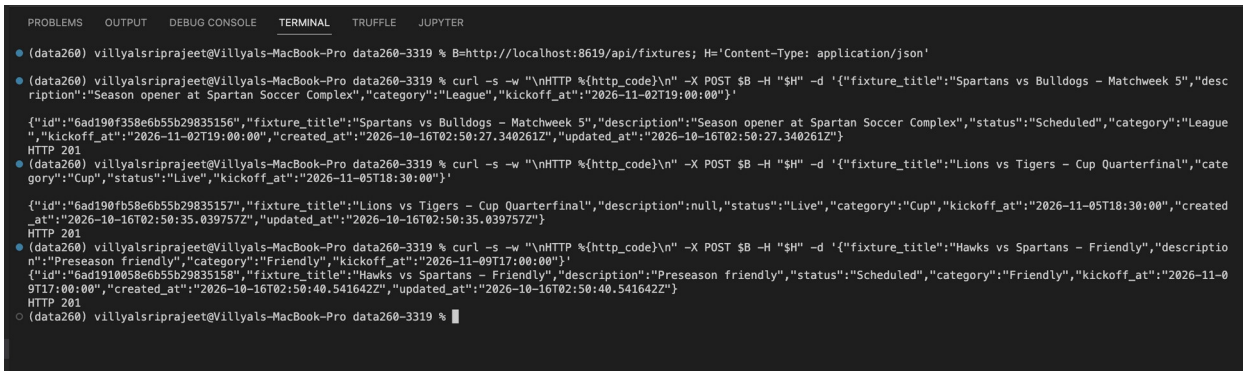
```
@router.put("/{fixture_id}", response_model=FixtureDocOut)
def update_fixture(fixture_id: str, payload: FixtureDocUpdate):
    oid = to_object_id(fixture_id)
    changes = payload.model_dump(exclude_unset=True)  # only the fields that were sent
    if not changes:
        raise HTTPException(status_code=400, detail="No fields provided to update")
    changes["updated_at"] = datetime.now(timezone.utc)
    doc = get_fixtures().find_one_and_update({"_id": oid}, {"$set": changes}, return_document=ReturnDocument.AFTER)
    if doc is None:
        raise HTTPException(status_code=404, detail=f"Fixture {fixture_id} not found")
    return serialize(doc)
@router.delete("/{fixture_id}")
def delete_fixture(fixture_id: str):
    if get_fixtures().delete_one({"_id": to_object_id(fixture_id)}).deleted_count == 0:
        raise HTTPException(status_code=404, detail=f"Fixture {fixture_id} not found")
    return {"message": "Fixture deleted", "id": fixture_id}
```

CORS and error handlers (code/api/main.py)

```python
app = FastAPI(title="Community Sports League Fixtures - HW6", lifespan=lifespan)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],  # the React dev server
    allow_credentials=True,  # allow the session cookie on cross origin calls
    allow_methods=["*"],
    allow_headers=["*"],
# ...
@app.exception_handler(WriteError)
async def mongo_write_error(request, exc):
    code = 422 if exc.code == 121 else 500  # 121 = the collection validator rejected the document
    return JSONResponse(status_code=code, content={"detail": f"Database rejected the write: {exc}"})
@app.exception_handler(ServerSelectionTimeoutError)
async def mongo_down(request, exc):
    return JSONResponse(status_code=503, content={"detail": "MongoDB is not reachable"})
@app.exception_handler(PyMongoError)
async def mongo_error(request, exc):
    return JSONResponse(status_code=500, content={"detail": "Database error"})
```

3. CRUD results

POST /api/fixtures - 201 Created (three fixtures)



GET all, GET one and PUT - 200 OK



DELETE - 200 OK, then GET the deleted fixture - 404 Not Found



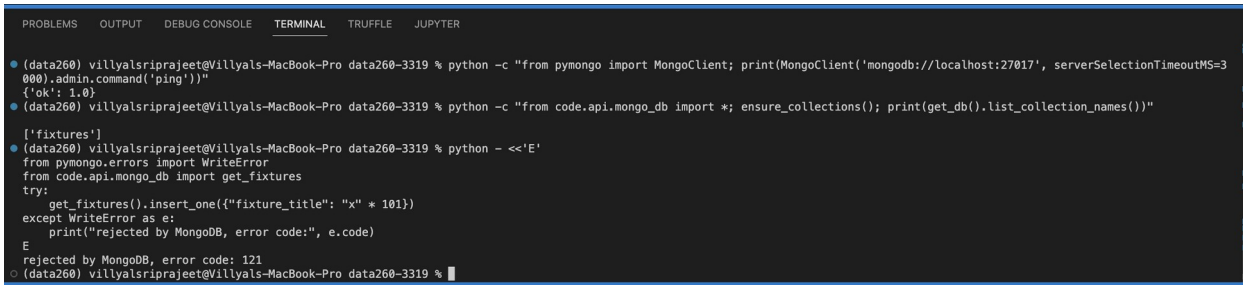
4. Validation tests

A missing required field, an invalid enum value, a wrong date and a wrong type all return 422. An id that is not a valid ObjectId returns 400.



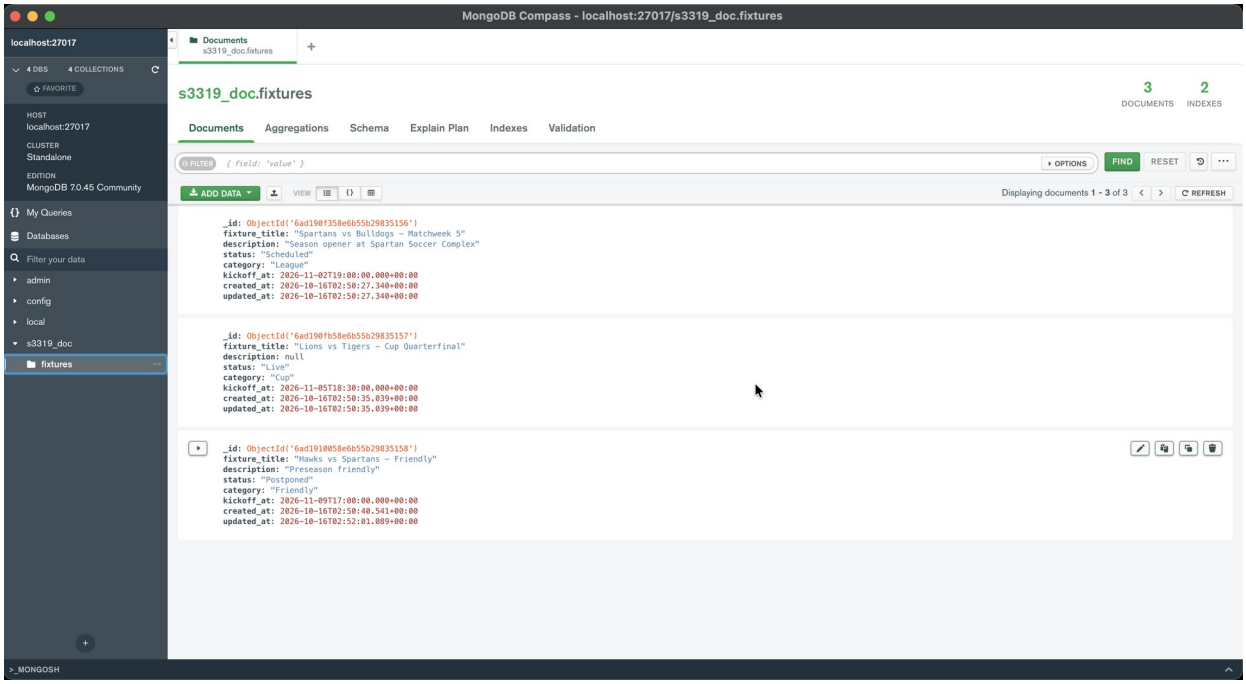
The collection validator also rejects bad data on its own. A 101-character title inserted straight through PyMongo is refused with error code 121.

Direct insert rejected by MongoDB (error code 121)

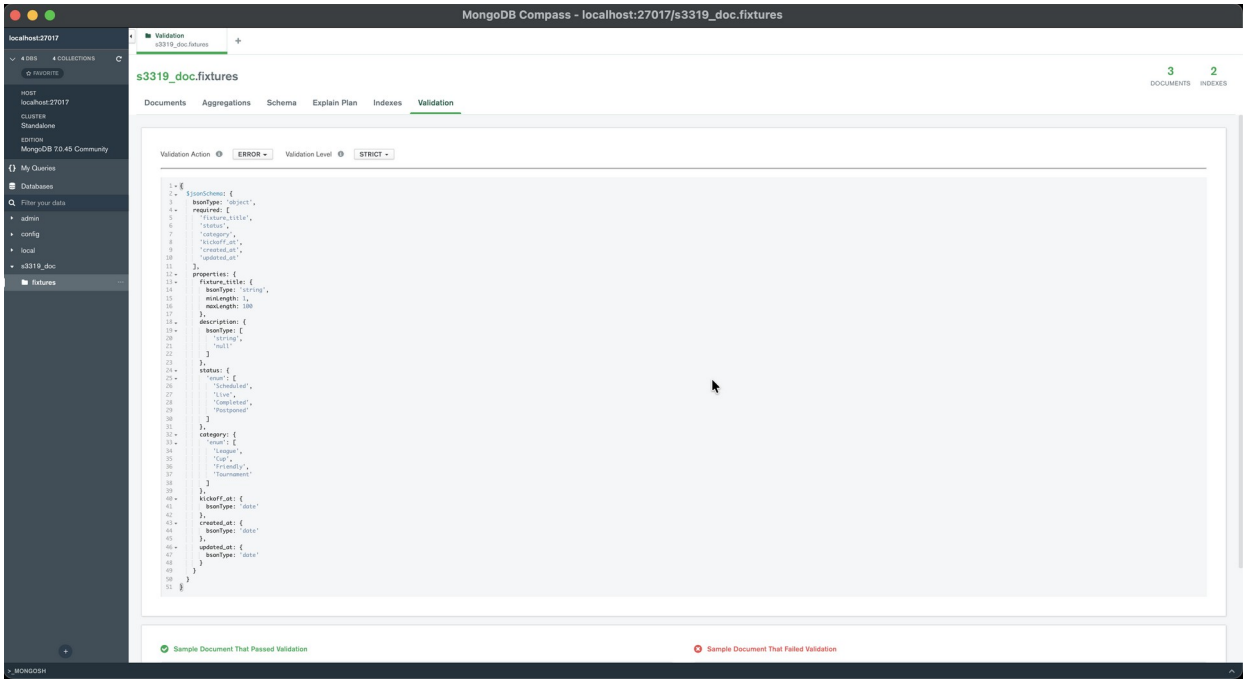


5. Compass

s3319_doc.fixtures - Documents tab



s3319_doc.fixtures - Validation tab



Part 2 – Chat Memory

1. Collections

Three new collections in the same database, each with a $jsonSchema validator and indexes on user_id and created_at.

Collection	Fields
messages	user_id, session_id, role (user or assistant), content, created_at
summaries	user_id, session_id (null for the lifetime summary), scope (session or user), text, created_at
episodes	user_id, session_id, fact (up to 300 characters), importance (0 to 1), embedding (384 floats), created_at

2. How POST /api/chat works

Each request saves the user message, then builds the prompt in this order: system primer, long-term summaries, short-term window, current message, episodic facts. The reply from qwen2.5:3b is saved, up to 3 facts are extracted from the user message and stored with an embedding, and a session summary is written every 5 user messages. After every 3 session summaries they are condensed into a lifetime summary (scope user, session_id null).

Setting	Value
SHORT_TERM_N (messages in the window)	12
SUMMARIZE_EVERY_USER_MSGS	5
Lifetime summary	after every 3 session summaries
Episodic retrieval	top 3 facts by cosine similarity
Embedding model	all-MiniLM-L6-v2 (384 dimensions)

3. Code

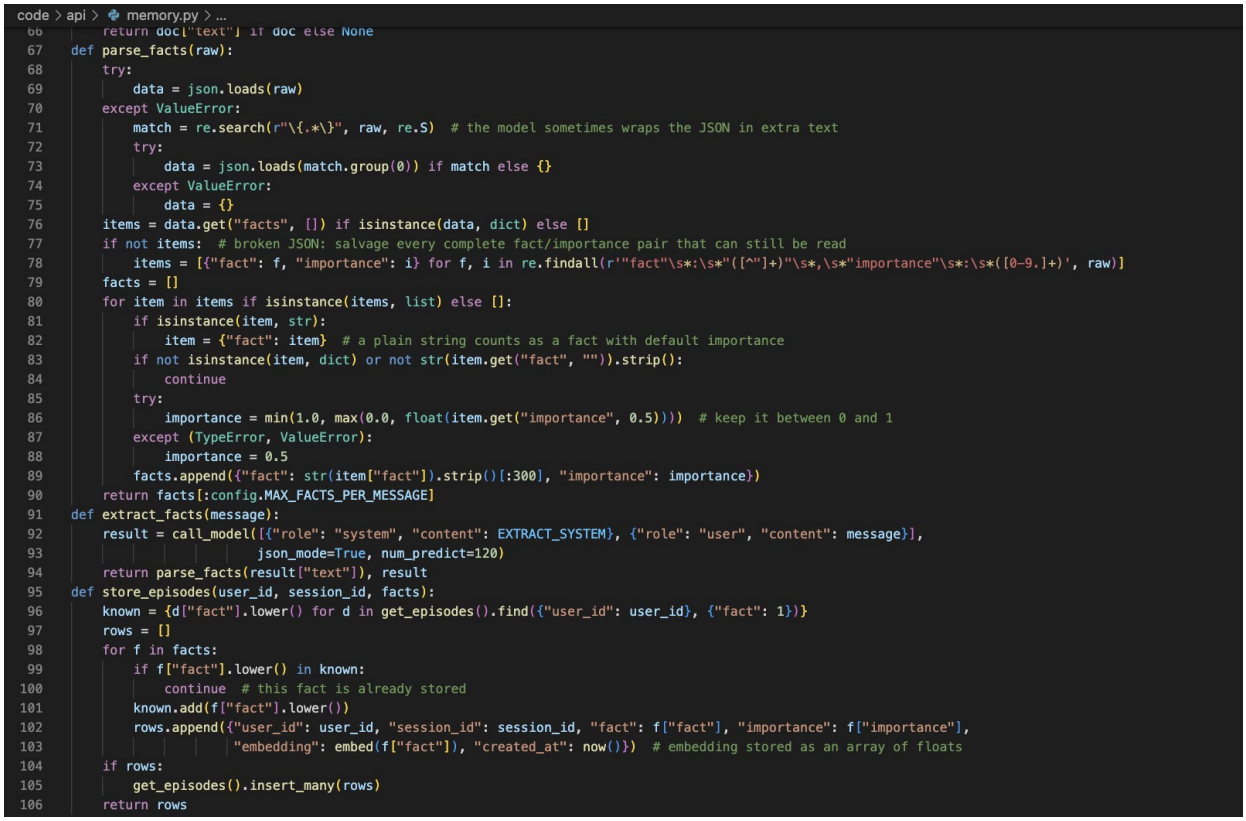
Endpoints: POST /api/chat, GET /api/memory/{user_id}, GET /api/aggregate/{user_id} (code/api/routers/chat.py)

code > api > routers > chat.py > ...

```python
# Part 2 endpoints: POST /api/chat, GET /api/memory/{user_id}, GET /api/aggregate/{user_id}
import uuid
from urllib.error import URLError
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel, ConfigDict, Field
from .. import memory
from ..mongo_db import get_episodes, get_messages, get_summaries
router = APIRouter(prefix="/api", tags=["Chat memory (MongoDB)"])
class ChatRequest(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    user_id: str = Field(min_length=1, max_length=64)
    session_id: str | None = Field(default=None, min_length=1, max_length=64)  # optional, defaults to the latest session
    message: str = Field(min_length=1, max_length=2000)
def clean(doc):
    doc = {k: v for k, v in doc.items() if k not in ("_id", "embedding")}  # no ObjectId and no 384-number vectors in the JSON
    if doc.get("created_at"):
        doc["created_at"] = doc["created_at"].isoformat()
    return doc
def latest_session_id(user_id):
    doc = get_messages().find_one({"user_id": user_id}, sort=[("created_at", -1), ("_id", -1)])
    return doc["session_id"] if doc else None
@router.post("/chat")
def chat(body: ChatRequest):
    session_id = body.session_id or latest_session_id(body.user_id) or f"sess-{uuid.uuid4().hex[:8]}"  # latest session, else a new one
    try:
        out = memory.chat_turn(body.user_id, session_id, body.message)
    except (URLError, ConnectionError) as exc:
        raise HTTPException(status_code=503, detail=f"Model server is not reachable: {exc}")
    return {"user_id": body.user_id, "session_id": session_id, "reply": out["reply"], "used": out["used"], "metrics": out["metrics"]}
@router.get("/memory/{user_id}")
def get_memory(user_id: str):
    session_id = latest_session_id(user_id)
    rows = list(get_messages().find({"user_id": user_id}).sort([("created_at", -1), ("_id", -1)]).limit(16))
    facts = list(get_episodes().find({"user_id": user_id}).sort([("created_at", -1), ("_id", -1)]).limit(20))
    session_doc = get_summaries().find_one({"user_id": user_id, "scope": "session", "session_id": session_id},
                                           sort=[("created_at", -1), ("_id", -1)]) if session_id else None
    lifetime_doc = get_summaries().find_one({"user_id": user_id, "scope": "user"}, sort=[("created_at", -1), ("_id", -1)])
    return {"user_id": user_id, "session_id": session_id,
            "recent_messages": [clean(r) for r in reversed(rows)],  # oldest first
            "session_summary": clean(session_doc) if session_doc else None,
            "lifetime_summary": clean(lifetime_doc) if lifetime_doc else None,
            "episodic_facts": [clean(f) for f in facts]}
@router.get("/aggregate/{user_id}")
def aggregate(user_id: str):
    pipeline = [{"$match": {"user_id": user_id}},
                {"$group": {"_id": {"$dateToString": {"format": "%Y-%m-%d", "date": "$created_at"}}, "count": {"$sum": 1}}},  # messages per day
                {"$sort": {"_id": 1}}]
    daily = [{"date": d["_id"], "count": d["count"]} for d in get_messages().aggregate(pipeline)]
    recent = list(get_summaries().find({"user_id": user_id}).sort([("created_at", -1), ("_id", -1)]).limit(5))
    return {"user_id": user_id, "daily_message_counts": daily, "recent_summaries": [clean(s) for s in recent]}

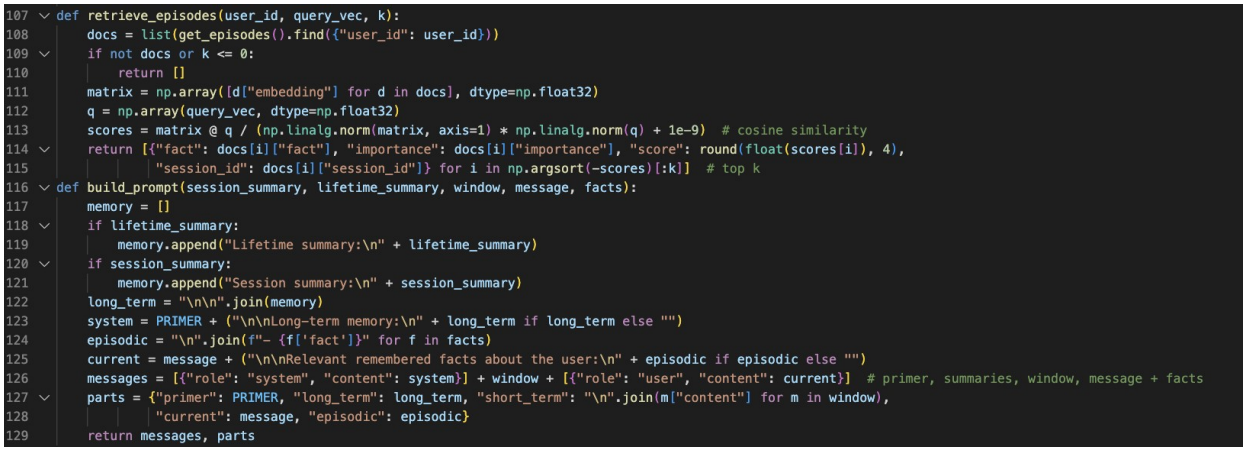
```

Episode extraction and storage (code/api/memory.py)



```
code > api > memory.py > ...
    return doc["text"] if doc else None
def parse_facts(raw):
    try:
        data = json.loads(raw)
    except ValueError:
        match = re.search(r"\{.*\}", raw, re.S)  # the model sometimes wraps the JSON in extra text
        try:
            data = json.loads(match.group(0)) if match else {}
        except ValueError:
            data = {}
    items = data.get("facts", []) if isinstance(data, dict) else []
    if not items:  # broken JSON: salvage every complete fact/importance pair that can still be read
        items = [{"fact": f, "importance": i} for f, i in re.findall(r'"fact"\s*:\s*"([^"]+)"\s*,\s*"importance"\s*:\s*([0-9.]+)', raw)]
    facts = []
    for item in items if isinstance(items, list) else []:
        if isinstance(item, str):
            item = {"fact": item}  # a plain string counts as a fact with default importance
        if not isinstance(item, dict) or not str(item.get("fact", "")).strip():
            continue
        try:
            importance = min(1.0, max(0.0, float(item.get("importance", 0.5))))  # keep it between 0 and 1
        except (TypeError, ValueError):
            importance = 0.5
        facts.append({"fact": str(item["fact"]).strip()[:300], "importance": importance})
    return facts[:config.MAX_FACTS_PER_MESSAGE]
def extract_facts(message):
    result = call_model([{"role": "system", "content": EXTRACT_SYSTEM}, {"role": "user", "content": message}],
                        json_mode=True, num_predict=120)
    return parse_facts(result["text"]), result
def store_episodes(user_id, session_id, facts):
    known = {d["fact"].lower() for d in get_episodes().find({"user_id": user_id}, {"fact": 1})}
    rows = []
    for f in facts:
        if f["fact"].lower() in known:
            continue  # this fact is already stored
        known.add(f["fact"].lower())
        rows.append({"user_id": user_id, "session_id": session_id, "fact": f["fact"], "importance": f["importance"],
                     "embedding": embed(f["fact"]), "created_at": now()})  # embedding stored as an array of floats
    if rows:
        get_episodes().insert_many(rows)
    return rows
```

Embedding retrieval and prompt assembly (code/api/memory.py)



```
def retrieve_episodes(user_id, query_vec, k):
    docs = list(get_episodes().find({"user_id": user_id}))
    if not docs or k <= 0:
        return []
    matrix = np.array([d["embedding"] for d in docs], dtype=np.float32)
    q = np.array(query_vec, dtype=np.float32)
    scores = matrix @ q / (np.linalg.norm(matrix, axis=1) * np.linalg.norm(q) + 1e-9)  # cosine similarity
    return [{"fact": docs[i]["fact"], "importance": docs[i]["importance"], "score": round(float(scores[i]), 4),
             "session_id": docs[i]["session_id"]} for i in np.argsort(-scores)[:k]]  # top k
def build_prompt(session_summary, lifetime_summary, window, message, facts):
    memory = []
    if lifetime_summary:
        memory.append("Lifetime summary:\n" + lifetime_summary)
    if session_summary:
        memory.append("Session summary:\n" + session_summary)
    long_term = "\n\n".join(memory)
    system = PRIMER + ("\n\nLong-term memory:\n" + long_term if long_term else "")
    episodic = "\n".join(f"- {f['fact']}" for f in facts)
    current = message + ("\n\nRelevant remembered facts about the user:\n" + episodic if episodic else "")
    messages = [{"role": "system", "content": system}] + window + [{"role": "user", "content": current}]  # primer, summaries, window, message + facts
    parts = {"primer": PRIMER, "long_term": long_term, "short_term": "\n".join(m["content"] for m in window),
             "current": message, "episodic": episodic}
    return messages, parts
```

Summarization (code/api/memory.py)



```
def summarize_session(user_id, session_id):
    last = get_summaries().find_one({"user_id": user_id, "scope": "session", "session_id": session_id},
                                    sort=[("created_at", -1), ("_id", -1)])
    query = {"user_id": user_id, "session_id": session_id}
    if last:
        query["created_at"] = {"$gt": last["created_at"]}  # only the messages since the last summary
    rows = list(get_messages().find(query).sort([("created_at", 1), ("_id", 1)]))
    text = "\n".join(f"{r['role']}: {r['content']}" for r in rows)
    prompt = f"Previous summary:\n{last['text'] if last else 'none'}\n\nNew messages:\n{text}\n\nWrite the updated summary."
    result = call_model([{"role": "system", "content": SUMMARY_SYSTEM}, {"role": "user", "content": prompt}], num_predict=200)
    doc = {"user_id": user_id, "session_id": session_id, "scope": "session", "text": result["text"].strip(), "created_at": now()}
    get_summaries().insert_one(doc)
    return result
def maybe_lifetime(user_id):
    sessions = get_summaries().find({"user_id": user_id, "scope": "session"})
    docs = list(sessions.sort([("created_at", -1), ("_id", -1)]))
    if not docs or len(docs) % config.LIFETIME_EVERY_SUMMARIES:
        return None  # only after every few session summaries
    recent = "\n\n".join(d["text"] for d in reversed(docs[:config.LIFETIME_EVERY_SUMMARIES]))
    prompt = f"Previous lifetime summary:\n{latest_summary(user_id, 'user') or 'none'}\n\nSession summaries:\n{recent}\n\nWrite the lifetime summary."
    result = call_model([{"role": "system", "content": LIFETIME_SYSTEM}, {"role": "user", "content": prompt}], num_predict=250)
    get_summaries().insert_one({"user_id": user_id, "session_id": None, "scope": "user", "text": result["text"].strip(),
                                "created_at": now()})  # session_id stays null for the lifetime summary
    return result
```

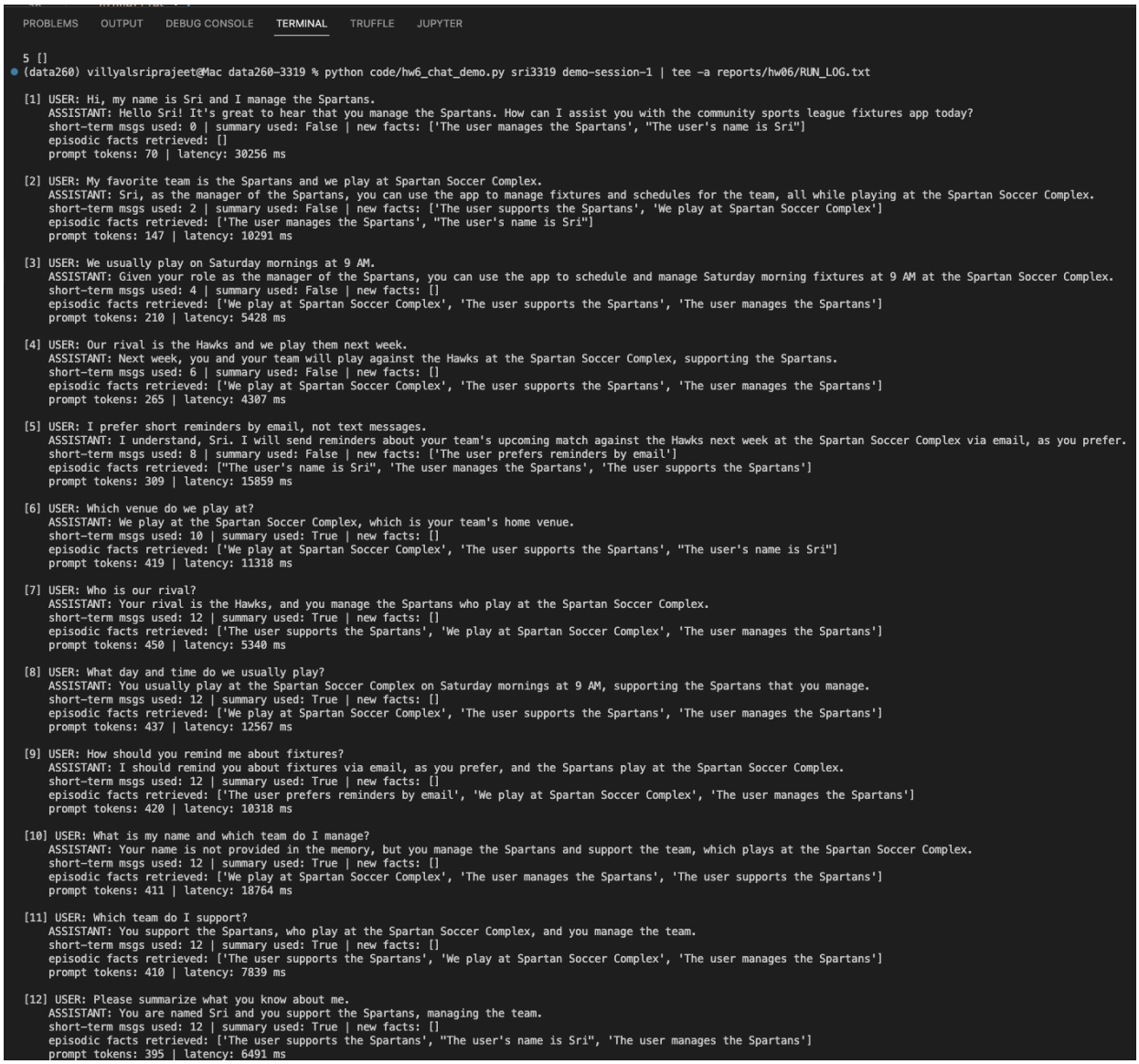
Full chat turn and the summarization trigger (code/api/memory.py)

```python
def chat_turn(user_id, session_id, message, short_term_n=None, summarize_every=None, top_k=None, log_path=config.METRICS_LOG):
    n = config.SHORT_TERM_N if short_term_n is None else short_term_n
    every = config.SUMMARIZE_EVERY_USER_MSGS if summarize_every is None else summarize_every
    k = config.EPISODIC_TOP_K if top_k is None else top_k
    started = time.perf_counter()
    saved = save_message(user_id, session_id, "user", message)  # 1. save the user message
    window = short_term(user_id, session_id, n, saved["_id"])  # 2. short-term window
    session_summary = latest_summary(user_id, "session", session_id)  # 3. summaries
    lifetime_summary = latest_summary(user_id, "user")
    facts = retrieve_episodes(user_id, embed(message), k)  # 4. top-k episodic facts
    messages, parts = build_prompt(session_summary, lifetime_summary, window, message, facts)  # 5. prompt
    t0 = time.perf_counter()
    result = call_model(messages)  # 6. model call
    reply_ms = round((time.perf_counter() - t0) * 1000)
    reply = result["text"].strip() or "(empty reply)"
    save_message(user_id, session_id, "assistant", reply)  # 7. save the reply
    new_facts, extract = extract_facts(message)  # 8. episodic extraction
    saved_facts = store_episodes(user_id, session_id, new_facts)
    user_count = get_messages().count_documents({"user_id": user_id, "session_id": session_id, "role": "user"})
    summary = None
    if every > 0 and user_count % every == 0:  # 9. summary every M user messages
        summary = summarize_session(user_id, session_id)
        lifetime = maybe_lifetime(user_id)
        if lifetime:
            summary = {"input_tokens": summary["input_tokens"] + lifetime["input_tokens"],
                       "output_tokens": summary["output_tokens"] + lifetime["output_tokens"]}
    metrics = {"timestamp": now().isoformat(timespec="seconds"), "user_id": user_id, "session_id": session_id, "turn": user_count,
               "short_term_n": n, "prompt_tokens": result["input_tokens"], "completion_tokens": result["output_tokens"],
               "primer_tokens": count_tokens(parts["primer"]), "long_term_tokens": count_tokens(parts["long_term"]),
               "short_term_tokens": count_tokens(parts["short_term"]), "current_tokens": count_tokens(parts["current"]),
               "episodic_tokens": count_tokens(parts["episodic"]), "extract_prompt_tokens": extract["input_tokens"],
               "extract_completion_tokens": extract["output_tokens"],
               "summary_prompt_tokens": summary["input_tokens"] if summary else 0,
               "summary_completion_tokens": summary["output_tokens"] if summary else 0,
               "latency_ms": round((time.perf_counter() - started) * 1000), "reply_latency_ms": reply_ms,
               "token_method": token_method()}
    if log_path:
        log_metrics(metrics, log_path)
    used = {"short_term_count": len(window), "short_term": window, "long_term_summary": parts["long_term"] or None,
            "episodic_facts": facts, "new_facts_saved": [r["fact"] for r in saved_facts], "summary_created": summary is not None}
    return {"reply": reply, "used": used, "metrics": metrics}

```

4. Results

Conversation: 12 messages in one session (demo-session-1)



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  TRUFFLE  JUPYTER

5 []
(data260) villyalsriprajeet@Mac data260-3319 % python code/hw6_chat_demo.py sri3319 demo-session-1 | tee -a reports/hw06/RUN_LOG.txt

[1] USER: Hi, my name is Sri and I manage the Spartans.
    ASSISTANT: Hello Sri! It's great to hear that you manage the Spartans. How can I assist you with the community sports league fixtures app today?
    short-term msgs used: 0 | summary used: False | new facts: ['The user manages the Spartans', "The user's name is Sri"]
    episodic facts retrieved: []
    prompt tokens: 70 | latency: 30256 ms

[2] USER: My favorite team is the Spartans and we play at Spartan Soccer Complex.
    ASSISTANT: Sri, as the manager of the Spartans, you can use the app to manage fixtures and schedules for the team, all while playing at the Spartan Soccer Complex.
    short-term msgs used: 2 | summary used: False | new facts: ['The user supports the Spartans', 'We play at Spartan Soccer Complex']
    episodic facts retrieved: ['The user manages the Spartans', "The user's name is Sri"]
    prompt tokens: 147 | latency: 10291 ms

[3] USER: We usually play on Saturday mornings at 9 AM.
    ASSISTANT: Given your role as the manager of the Spartans, you can use the app to schedule and manage Saturday morning fixtures at 9 AM at the Spartan Soccer Complex.
    short-term msgs used: 4 | summary used: False | new facts: []
    episodic facts retrieved: ['We play at Spartan Soccer Complex', 'The user supports the Spartans', 'The user manages the Spartans']
    prompt tokens: 210 | latency: 5428 ms

[4] USER: Our rival is the Hawks and we play them next week.
    ASSISTANT: Next week, you and your team will play against the Hawks at the Spartan Soccer Complex, supporting the Spartans.
    short-term msgs used: 6 | summary used: False | new facts: []
    episodic facts retrieved: ['We play at Spartan Soccer Complex', 'The user supports the Spartans', 'The user manages the Spartans']
    prompt tokens: 265 | latency: 4307 ms

[5] USER: I prefer short reminders by email, not text messages.
    ASSISTANT: I understand, Sri. I will send reminders about your team's upcoming match against the Hawks next week at the Spartan Soccer Complex via email, as you prefer.
    short-term msgs used: 8 | summary used: False | new facts: ['The user prefers reminders by email']
    episodic facts retrieved: ["The user's name is Sri", 'The user manages the Spartans', 'The user supports the Spartans']
    prompt tokens: 309 | latency: 15859 ms

[6] USER: Which venue do we play at?
    ASSISTANT: We play at the Spartan Soccer Complex, which is your team's home venue.
    short-term msgs used: 10 | summary used: True | new facts: []
    episodic facts retrieved: ['We play at Spartan Soccer Complex', 'The user supports the Spartans', "The user's name is Sri"]
    prompt tokens: 419 | latency: 11318 ms

[7] USER: Who is our rival?
    ASSISTANT: Your rival is the Hawks, and you manage the Spartans who play at the Spartan Soccer Complex.
    short-term msgs used: 12 | summary used: True | new facts: []
    episodic facts retrieved: ['The user supports the Spartans', 'We play at Spartan Soccer Complex', 'The user manages the Spartans']
    prompt tokens: 450 | latency: 5340 ms

[8] USER: What day and time do we usually play?
    ASSISTANT: You usually play at the Spartan Soccer Complex on Saturday mornings at 9 AM, supporting the Spartans that you manage.
    short-term msgs used: 12 | summary used: True | new facts: []
    episodic facts retrieved: ['We play at Spartan Soccer Complex', 'The user supports the Spartans', 'The user manages the Spartans']
    prompt tokens: 437 | latency: 12567 ms

[9] USER: How should you remind me about fixtures?
    ASSISTANT: I should remind you about fixtures via email, as you prefer, and the Spartans play at the Spartan Soccer Complex.
    short-term msgs used: 12 | summary used: True | new facts: []
    episodic facts retrieved: ['The user prefers reminders by email', 'We play at Spartan Soccer Complex', 'The user manages the Spartans']
    prompt tokens: 420 | latency: 10318 ms

[10] USER: What is my name and which team do I manage?
    ASSISTANT: Your name is not provided in the memory, but you manage the Spartans and support the team, which plays at the Spartan Soccer Complex.
    short-term msgs used: 12 | summary used: True | new facts: []
    episodic facts retrieved: ['We play at Spartan Soccer Complex', 'The user manages the Spartans', 'The user supports the Spartans']
    prompt tokens: 411 | latency: 18764 ms

[11] USER: Which team do I support?
    ASSISTANT: You support the Spartans, who play at the Spartan Soccer Complex, and you manage the team.
    short-term msgs used: 12 | summary used: True | new facts: []
    episodic facts retrieved: ['The user supports the Spartans', 'We play at Spartan Soccer Complex', 'The user manages the Spartans']
    prompt tokens: 410 | latency: 7839 ms

[12] USER: Please summarize what you know about me.
    ASSISTANT: You are named Sri and you support the Spartans, managing the team.
    short-term msgs used: 12 | summary used: True | new facts: []
    episodic facts retrieved: ['The user supports the Spartans', "The user's name is Sri", 'The user manages the Spartans']
    prompt tokens: 395 | latency: 6491 ms
```

On message 10 the fact "The user's name is Sri" was not among the 3 facts retrieved, so the model answered that the name was not provided. Message 12 retrieved it and answered correctly.

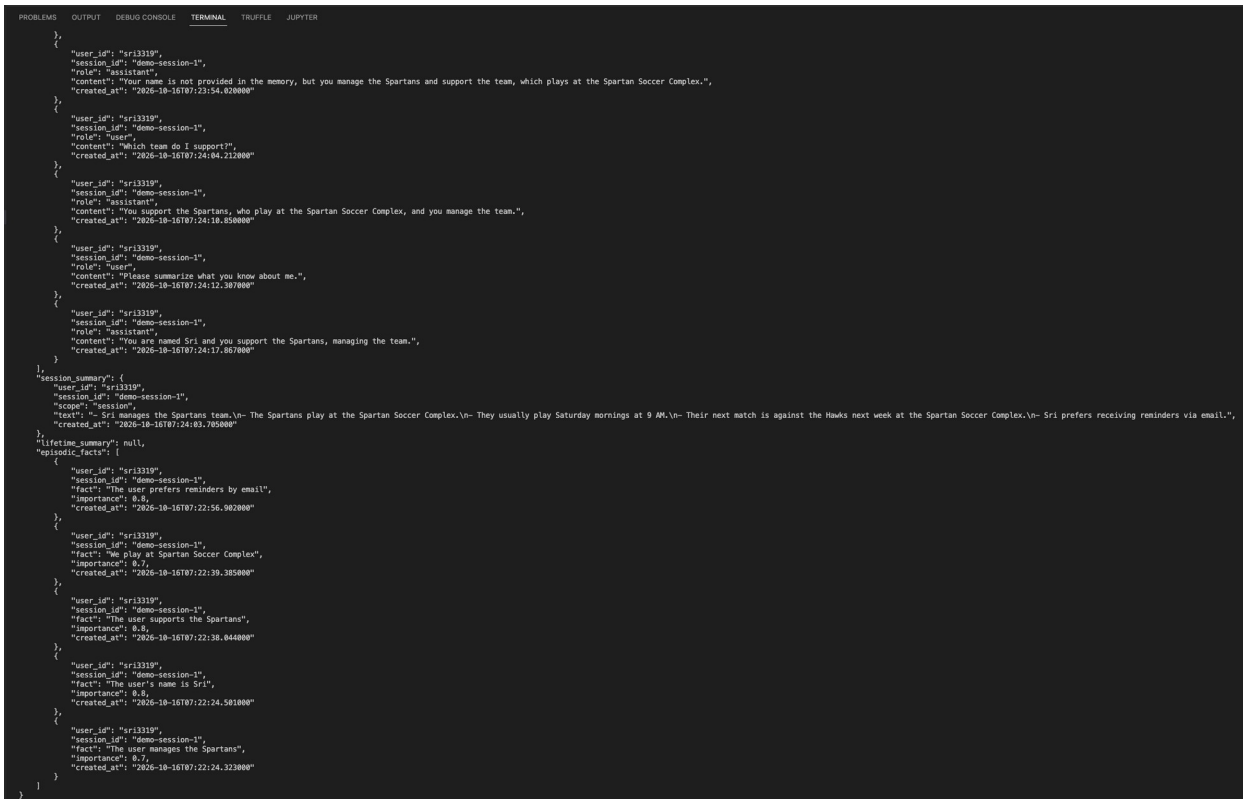
GET /api/memory/sri3319



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  TRUFFLE  JUPYTER

    prompt tokens: 395 | latency: 6491 ms
(data260) villyalsriprajeet@Mac data260-3319 % curl -s http://127.0.0.1:8619/api/memory/sri3319 | python -m json.tool | tee -a reports/hw06/RUN_LOG.txt

{
    "user_id": "sri3319",
    "session_id": "demo-session-1",
    "recent_messages": [
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "user",
            "content": "I prefer short reminders by email, not text messages.",
            "created_at": "2026-10-16T07:22:49.464000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "assistant",
            "content": "I understand, Sri. I will send reminders about your team's upcoming match against the Hawks next week at the Spartan Soccer Complex via email, as you prefer.",
            "created_at": "2026-10-16T07:22:54.163000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "user",
            "content": "Which venue do we play at?",
            "created_at": "2026-10-16T07:23:05.428000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "assistant",
            "content": "We play at the Spartan Soccer Complex, which is your team's home venue.",
            "created_at": "2026-10-16T07:23:14.118000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "user",
            "content": "Who is our rival?",
            "created_at": "2026-10-16T07:23:16.832000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "assistant",
            "content": "Your rival is the Hawks, and you manage the Spartans who play at the Spartan Soccer Complex.",
            "created_at": "2026-10-16T07:23:20.975000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "user",
            "content": "What day and time do we usually play?",
            "created_at": "2026-10-16T07:23:22.243000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "assistant",
            "content": "You usually play at the Spartan Soccer Complex on Saturday mornings at 9 AM, supporting the Spartans that you manage.",
            "created_at": "2026-10-16T07:23:33.619000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "user",
            "content": "How should you remind me about fixtures?",
            "created_at": "2026-10-16T07:23:34.933000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "assistant",
            "content": "I should remind you about fixtures via email, as you prefer, and the Spartans play at the Spartan Soccer Complex.",
            "created_at": "2026-10-16T07:23:44.051000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "user",
            "content": "What is my name and which team do I manage?",
            "created_at": "2026-10-16T07:23:45.355000"
        },
        {
            "user_id": "sri3319",
```

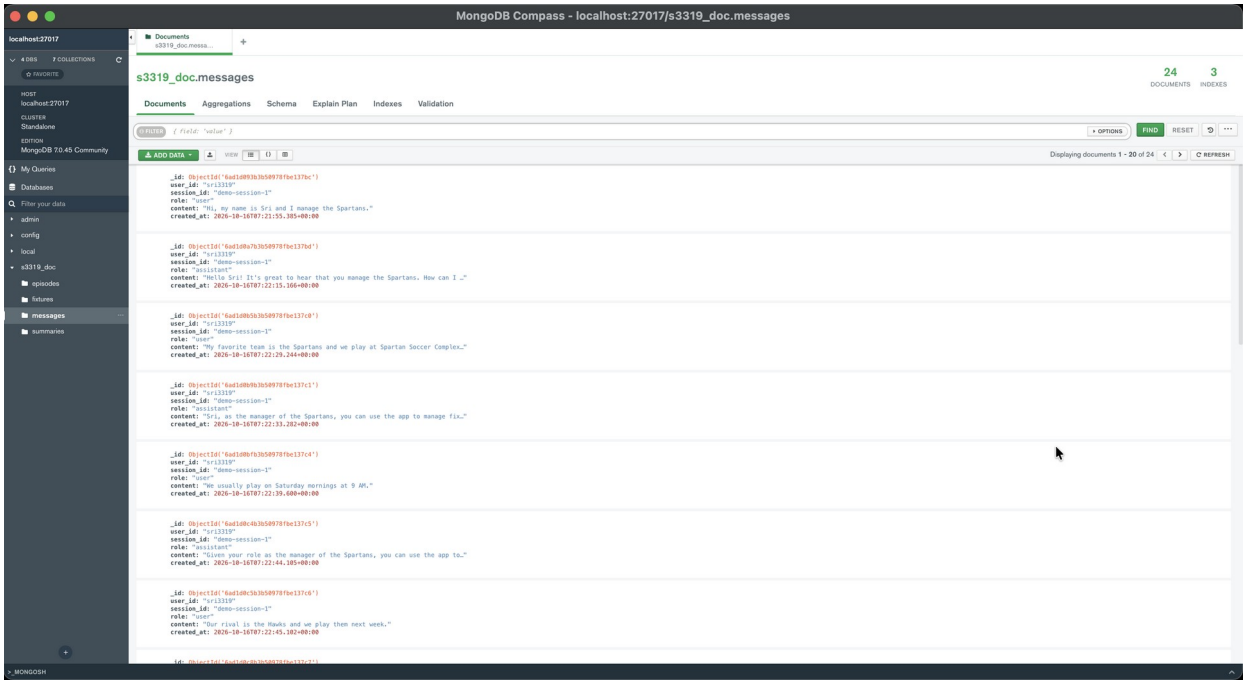



```
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "assistant",
            "content": "Your name is not provided in the memory, but you manage the Spartans and support the team, which plays at the Spartan Soccer Complex.",
            "created_at": "2026-10-16T07:23:54.020000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "user",
            "content": "Which team do I support?",
            "created_at": "2026-10-16T07:24:04.212000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "assistant",
            "content": "You support the Spartans, who play at the Spartan Soccer Complex, and you manage the team.",
            "created_at": "2026-10-16T07:24:10.850000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "user",
            "content": "Please summarize what you know about me.",
            "created_at": "2026-10-16T07:24:12.307000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "assistant",
            "content": "You are named Sri and you support the Spartans, managing the team.",
            "created_at": "2026-10-16T07:24:17.867000"
        }
    ],
    "session_summary": {
        "user_id": "sri3319",
        "session_id": "demo-session-1",
        "scope": "session",
        "text": "- Sri manages the Spartans team.\n- The Spartans play at the Spartan Soccer Complex.\n- They usually play Saturday mornings at 9 AM.\n- Their next match is against the Hawks next week at the Spartan Soccer Complex.\n- Sri prefers receiving reminders via email.",
        "created_at": "2026-10-16T07:24:03.705000"
    },
    "lifetime_summary": null,
    "episodic_facts": [
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "fact": "The user prefers reminders by email",
            "importance": 0.8,
            "created_at": "2026-10-16T07:22:56.902000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "fact": "We play at Spartan Soccer Complex",
            "importance": 0.7,
            "created_at": "2026-10-16T07:22:39.385000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "fact": "The user supports the Spartans",
            "importance": 0.8,
            "created_at": "2026-10-16T07:22:38.044000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "fact": "The user's name is Sri",
            "importance": 0.8,
            "created_at": "2026-10-16T07:22:24.501000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "fact": "The user manages the Spartans",
            "importance": 0.7,
            "created_at": "2026-10-16T07:22:24.323000"
        }
    ]
}
```

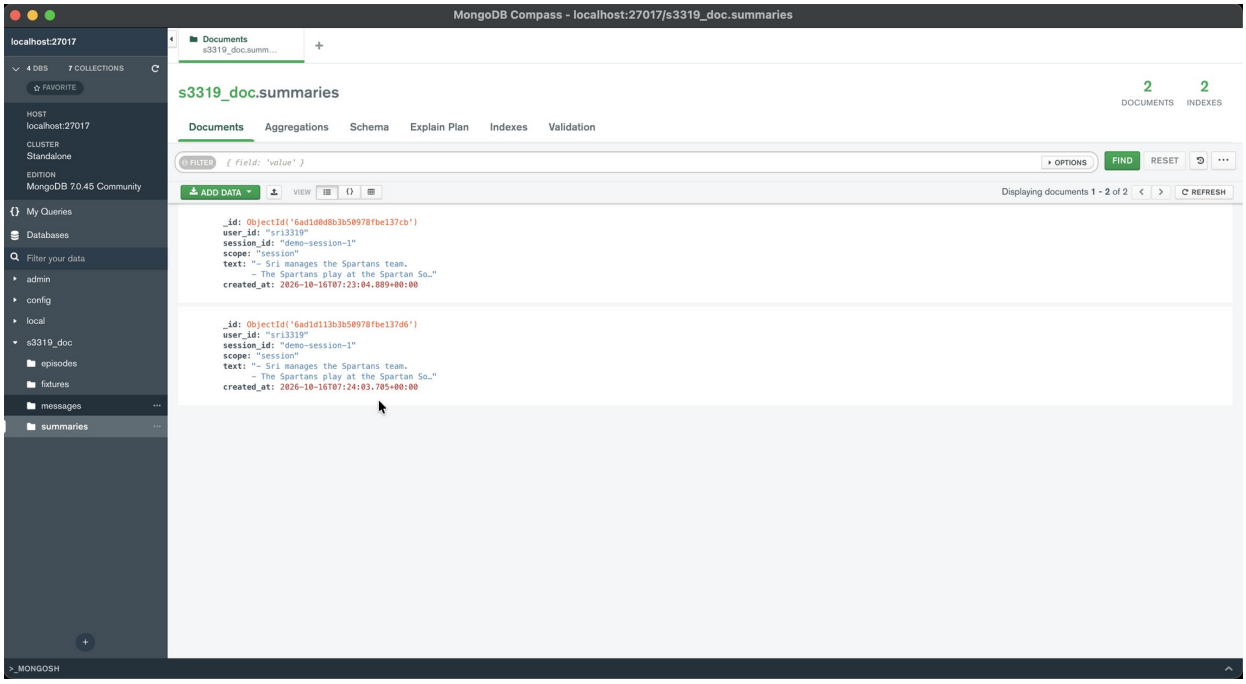
GET /api/aggregate/sri3319

```
(data260) villyalsriprajeet@Mac data260-3319 % curl -s http://127.0.0.1:8619/api/aggregate/sri3319 | python -m json.tool | tee -a reports/hw06/RUN_LOG.txt
{
    "user_id": "sri3319",
    "daily_message_counts": [
        {
            "date": "2026-10-16",
            "count": 24
        }
    ],
    "recent_summaries": [
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "scope": "session",
            "text": "- Sri manages the Spartans team.\n- The Spartans play at the Spartan Soccer Complex.\n- They usually play Saturday mornings at 9 AM.\n- Their next match is against the Hawks next week at the Spartan Soccer Complex.\n- Sri prefers receiving reminders via email.",
            "created_at": "2026-10-16T07:24:03.705000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "scope": "session",
            "text": "- Sri manages the Spartans team.\n- The Spartans play at the Spartan Soccer Complex.\n- They usually play Saturday mornings at 9 AM.\n- Their next match is against the Hawks next week at the Spartan Soccer Complex.\n- Sri prefers receiving reminders via email.",
            "created_at": "2026-10-16T07:23:04.889000"
        }
    ]
}
(data260) villyalsriprajeet@Mac data260-3319 %
```

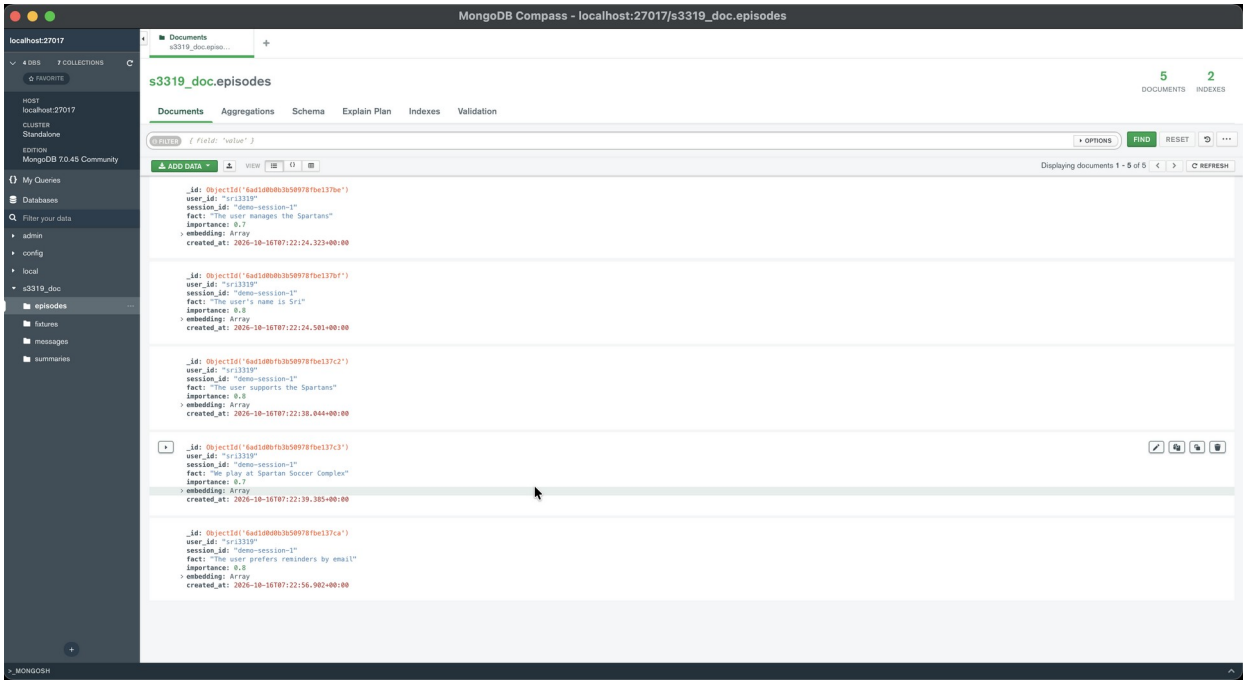
Compass: s3319_doc.messages (24 documents)



Compass: s3319_doc.summaries (2 documents)



Compass: s3319_doc.episodes (5 documents)



Part 3 – What Memory Costs

1. Logging memory usage

Every chat turn writes one row to a CSV: prompt and completion tokens, the tokens used by each memory part (system primer, long-term summary, short-term window, current message, episodic facts), the extra tokens spent on fact extraction and summaries, and the latency. The cost estimate uses the token counts reported by the model. I also count each part with the Qwen tokenizer, which comes out about 15% lower because Ollama counts the chat template tokens as well.

Logging fields and writer (code/api/memory.py)

```python
METRIC_FIELDS = ["timestamp", "user_id", "session_id", "turn", "short_term_n", "prompt_tokens", "completion_tokens",
                 "primer_tokens", "long_term_tokens", "short_term_tokens", "current_tokens", "episodic_tokens",
                 "extract_prompt_tokens", "extract_completion_tokens", "summary_prompt_tokens",
                         "summary_completion_tokens",
                 "latency_ms", "reply_latency_ms", "token_method"]
# ...
def log_metrics(metrics, path):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    new_file = not path.exists()
    with open(path, "a", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=METRIC_FIELDS, extrasaction="ignore")
        if new_file:
            writer.writeheader()
        writer.writerow(metrics)
```

First rows of reports/hw06/raw/sweep_turns.csv

```
(data260) villyalsriprajeet@Mac data260-3319 % head -8 reports/hw06/raw/sweep_turns.csv | cut -d, -f2,4-12,17 | column -s, -t
user_id   turn  short_term_n  prompt_tokens  completion_tokens  primer_tokens  long_term_tokens  short_term_tokens  current_tokens  episodic_tokens  latency_ms
sweep-n4  1     4             70             30                 44             0                 0                  13              0                11882
sweep-n4  2     4             143            16                 44             0                 42                 11              15               7402
sweep-n4  3     4             191            15                 44             0                 68                 11              27               3843
sweep-n4  4     4             175            21                 44             0                 51                 12              27               4374
sweep-n4  5     4             180            29                 44             0                 57                 11              27               16544
sweep-n4  6     4             264            26                 44             63                71                 9               32               6697
sweep-n4  7     4             269            57                 44             63                73                 12              32               9694
(data260) villyalsriprajeet@Mac data260-3319 %
```

Raw result files in reports/hw06/raw

```
(data260) villyalsriprajeet@Mac data260-3319 % ls -l reports/hw06/raw reports/hw06
reports/hw06:
total 0
-rw-r--r--  1 villyalsriprajeet  staff   1950 16 Oct 11:43 memory_probes.yaml
drwxr-xr-x  9 villyalsriprajeet  staff    288 16 Oct 17:46 raw
-rw-r--r--  1 villyalsriprajeet  staff  29821 16 Oct 17:46 RUN_LOG.txt

reports/hw06/raw:
total 64
-rw-r--r--  1 villyalsriprajeet  staff   1897 16 Oct 00:24 chat_metrics.csv
-rw-r--r--  1 villyalsriprajeet  staff   3082 16 Oct 16:44 probe_results_run1_before_fix.json
-rw-r--r--  1 villyalsriprajeet  staff  13257 16 Oct 17:46 probe_results.json
-rw-r--r--  1 villyalsriprajeet  staff    390 16 Oct 16:44 sweep_summary_run1_before_fix.json
-rw-r--r--  1 villyalsriprajeet  staff   1556 16 Oct 17:46 sweep_summary.json
-rw-r--r--  1 villyalsriprajeet  staff   3859 16 Oct 16:44 sweep_turns_run1_before_fix.csv
-rw-r--r--  1 villyalsriprajeet  staff  15058 16 Oct 17:45 sweep_turns.csv
(data260) villyalsriprajeet@Mac data260-3319 %
```

2. Memory probes

memory_probes.yaml holds five fixed questions, the expected answer for each, a 1 to 5 rubric and the scoring rule. An answer is scored by the expected items it contains (for example "Riverside" and "Field 3"), and 5 means all of them are there. The file was committed before any test run, which the commit times show.

memory_probes.yaml

reports > hw06 > ! memory_probes.yaml

```yaml
# HW6 Part 3 memory probes, written and committed BEFORE any sweep run
version: 1
planted_facts:  # facts the fixed 30-message conversation plants (user message number)
  - {message: 1, fact: "name is Sri, manages the Spartans"}
  - {message: 2, fact: "home games at Riverside Park Field 3"}
  - {message: 5, fact: "usual kickoff is Saturday at 9 AM (later replaced by the message 26 update)"}
  - {message: 12, fact: "remind by email, never by text"}
  - {message: 19, fact: "biggest rival is the Hawks"}
  - {message: 26, fact: "next game moved to Sunday at 2 PM"}
rubric:  # quality score 1 to 5 for each answer
  5: "every expected item is stated and nothing contradicts the conversation"
  4: "correct but one expected item is missing"
  3: "about half of the expected items are right"
  2: "mostly wrong or vague, or a wrong fact is mixed in with a right one"
  1: "wrong, contradicts the conversation, or says it does not know"
scoring: "score = 1 + round(4 * matched_items / total_items); if any must_not word appears the score is at most 2"
probes:
  - id: P1
    question: "What is my name and which team do I manage?"
    expected: "Sri, the Spartans"
    must_include: [[sri], [spartans]]  # each inner list is one item, any word in it counts as a match
    must_not: []
  - id: P2
    question: "Where do our home games take place?"
    expected: "Riverside Park Field 3"
    must_include: [[riverside], ["field 3", "field three"]]
    must_not: []
  - id: P3
    question: "Which team is our biggest rival?"
    expected: "the Hawks"
    must_include: [[hawks]]
    must_not: []
  - id: P4
    question: "How do I want to be reminded about fixtures?"
    expected: "by email, not by text"
    must_include: [[email]]
    must_not: []
  - id: P5
    question: "When is our next game now?"
    expected: "Sunday at 2 PM (moved from the usual Saturday 9 AM)"
    must_include: [[sunday], ["2 pm", "2pm", "2:00", "14:00"]]
    must_not: []

```

Commit order: probes (11:44) before the runner (16:36)

```
(data260) villyalsriprajeet@Mac data260-3319 % git log --format='%h  %ci  %s' -4
21b95fc  2026-10-16 16:36:06 -0700  HW6 part 3: eviction policy and sweep runner
83f7a8c  2026-10-16 11:44:21 -0700  HW6 part 3
ca1ef4a  2026-10-16 11:43:11 -0700  HW6 part 2
aad300c  2026-10-16 01:02:51 -0700  HW6 parts 1 and 2: Mongo CRUD, chat memor
(data260) villyalsriprajeet@Mac data260-3319 %
```

3. Comparing short-term memory sizes

The same 30 user messages go into a fresh user and session for each setting. Messages 1, 2, 5, 12, 19 and 26 plant the facts (name and team, venue, usual kickoff, reminder preference, rival, and a moved game); the rest are league questions. Temperature is 0 and the seed is 3319. After message 30 the five probes are asked and nothing from them is saved. N counts stored messages, so N=30 covers the last 15 turns.

Fixed conversation, probe scoring (code/hw6_sweep.py)

```
PLANTED = {1: "Hi, my name is Sri and I manage the Spartans.", 2: "Our home games are at Riverside Park Field 3.",
           5: "We usually kick off on Saturdays at 9 AM.", 12: "Please remind me about fixtures by email, never by
                text.",
           19: "Our biggest rival is the Hawks.", 26: "Update: our next game has been moved to Sunday at 2 PM."}
def conversation():
    filler = iter(FILLER)
    return [PLANTED.get(i) or next(filler) for i in range(1, 31)]  # 30 user messages, facts planted at fixed
            positions
# ...
def score(answer, probe):
    text = answer.lower()
    hit = lambda w: re.search(r"\b" + re.escape(str(w).lower()) + r"\b", text) is not None
    groups = probe["must_include"]
    matched = sum(any(hit(w) for w in g) for g in groups)
    value = 1 if matched == 0 else 1 + round(4 * matched / len(groups))
    if any(hit(w) for w in probe.get("must_not") or []):
        value = min(value, 2)
    return value, matched
```

One setting: 30 turns, then the probes (code/hw6_sweep.py)

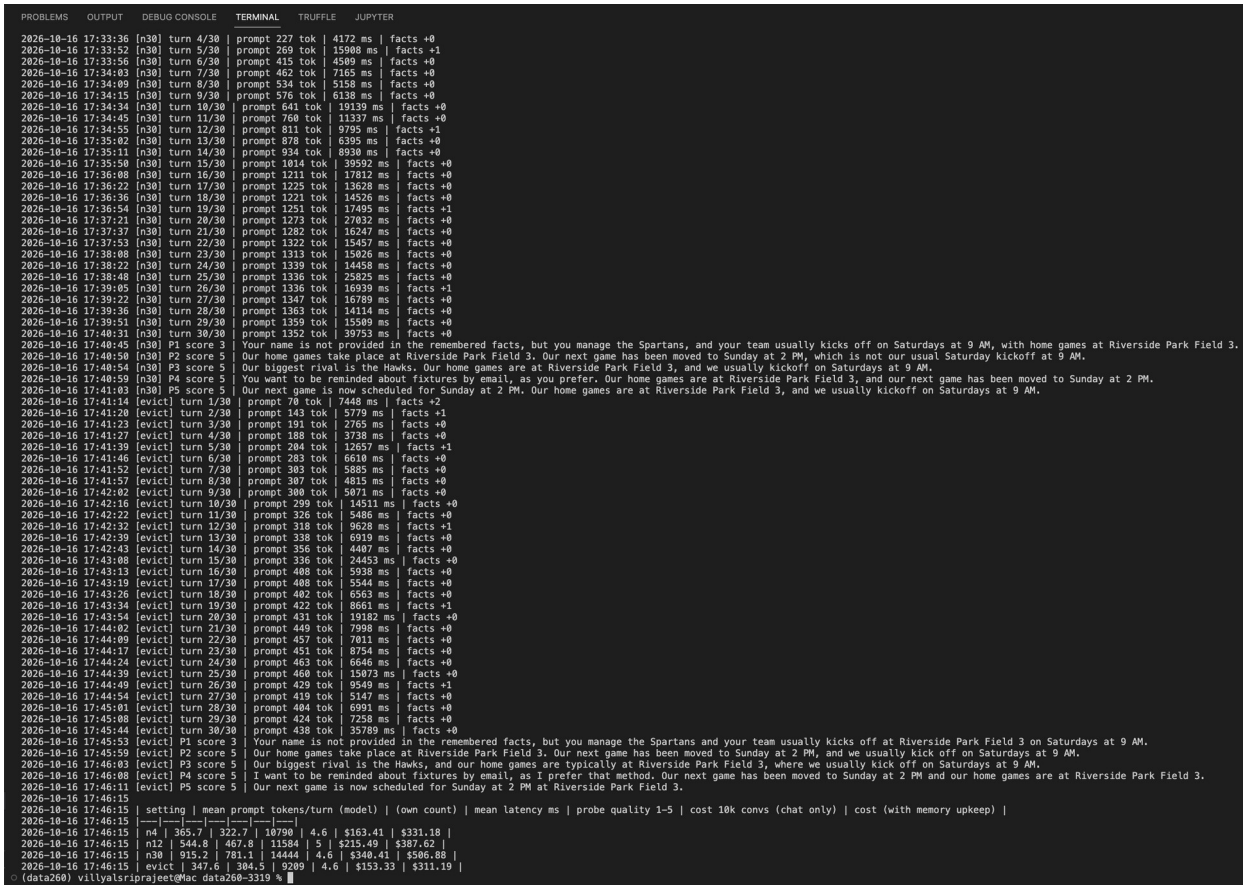
```
def run_setting(label, results):
    n, policy = SETTINGS[label]
    user_id, session_id = f"sweep-{label}", f"sweep-{label}-session"
    clear_user(user_id)
    memory.call_model([{"role": "user", "content": "Say hi."}], num_predict=5)  # load the model, embedder and
            tokenizer before timing starts
    embed("warm up")
    memory.count_tokens("warm up")
    for i, text in enumerate(conversation(), 1):
        out = memory.chat_turn(user_id, session_id, text, short_term_n=n, log_path=TURNS_CSV, policy=policy)
        print(f"[{label}] turn {i}/30 | prompt {out['metrics']['prompt_tokens']} tok | {out['metrics']['latency_ms']}
                ms | facts +{len(out['used']['new_facts_saved'])}", flush=True)
    probes = []
    for p in PROBES:
        r = ask_probe(user_id, session_id, p["question"], n, policy)
        r["score"], r["matched"] = score(r["answer"], p)
        r["id"], r["question"], r["expected"] = p["id"], p["question"], p["expected"]
        probes.append(r)
        print(f"[{label}] {p['id']} score {r['score']} | {r['answer']}", flush=True)
    results[label] = {"short_term_n": n, "policy": policy, "probes": probes, "mean_probe_quality":
            round(statistics.mean(r["score"] for r in probes), 2)}
    json.dump(results, open(RAW / "probe_results.json", "w"), indent=2)
```

Run output: N=4 and N=12

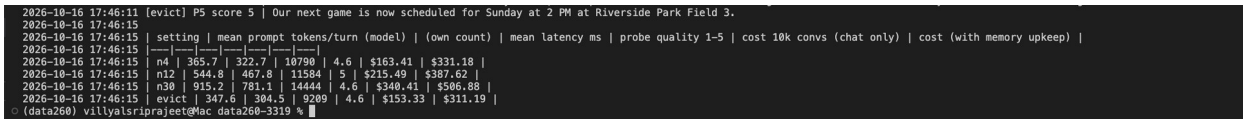
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  TRUFFLE  JUPYTER

(data260) villyalsriprajeet@Mac data260-3319 % caffeinate -i python -u -m code.hw6_sweep 2>&1 | while IFS= read -r l; do echo "$(date '+%Y-%m-%d %H:%M:%S') $l"; done | tee -a reports/hw06/RUN_LOG.txt
2026-10-16 17:21:03 /Users/villyalsriprajeet/miniconda3_x86/envs/data260/lib/python3.12/site-packages/huggingface_hub/file_download.py:1132: FutureWarning: `resume_download` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use `force_download=True`.
2026-10-16 17:21:03   warnings.warn(
2026-10-16 17:21:14 Special tokens have been added in the vocabulary, make sure the associated word embeddings are fine-tuned or trained.
2026-10-16 17:21:26 [n4] turn 1/30 | prompt 70 tok | 11882 ms | facts +2
2026-10-16 17:21:33 [n4] turn 2/30 | prompt 143 tok | 7402 ms | facts +1
2026-10-16 17:21:37 [n4] turn 3/30 | prompt 191 tok | 3843 ms | facts +0
2026-10-16 17:21:41 [n4] turn 4/30 | prompt 175 tok | 4374 ms | facts +0
2026-10-16 17:21:58 [n4] turn 5/30 | prompt 180 tok | 16544 ms | facts +1
2026-10-16 17:22:05 [n4] turn 6/30 | prompt 264 tok | 6697 ms | facts +0
2026-10-16 17:22:14 [n4] turn 7/30 | prompt 269 tok | 9694 ms | facts +0
2026-10-16 17:22:21 [n4] turn 8/30 | prompt 295 tok | 6202 ms | facts +0
2026-10-16 17:22:27 [n4] turn 9/30 | prompt 296 tok | 6521 ms | facts +0
2026-10-16 17:22:49 [n4] turn 10/30 | prompt 270 tok | 21519 ms | facts +0
2026-10-16 17:22:56 [n4] turn 11/30 | prompt 351 tok | 7567 ms | facts +0
2026-10-16 17:23:08 [n4] turn 12/30 | prompt 342 tok | 11600 ms | facts +1
2026-10-16 17:23:14 [n4] turn 13/30 | prompt 343 tok | 6189 ms | facts +0
2026-10-16 17:23:20 [n4] turn 14/30 | prompt 365 tok | 5416 ms | facts +0
2026-10-16 17:23:59 [n4] turn 15/30 | prompt 351 tok | 39826 ms | facts +0
2026-10-16 17:24:08 [n4] turn 16/30 | prompt 473 tok | 8705 ms | facts +0
2026-10-16 17:24:14 [n4] turn 17/30 | prompt 486 tok | 5578 ms | facts +0
2026-10-16 17:24:24 [n4] turn 18/30 | prompt 473 tok | 9734 ms | facts +0
2026-10-16 17:24:33 [n4] turn 19/30 | prompt 508 tok | 9132 ms | facts +1
2026-10-16 17:24:53 [n4] turn 20/30 | prompt 522 tok | 19781 ms | facts +0
2026-10-16 17:25:02 [n4] turn 21/30 | prompt 474 tok | 9318 ms | facts +0
2026-10-16 17:25:09 [n4] turn 22/30 | prompt 499 tok | 7127 ms | facts +0
2026-10-16 17:25:17 [n4] turn 23/30 | prompt 473 tok | 7419 ms | facts +0
2026-10-16 17:25:22 [n4] turn 24/30 | prompt 457 tok | 5216 ms | facts +0
2026-10-16 17:25:38 [n4] turn 25/30 | prompt 443 tok | 16558 ms | facts +0
2026-10-16 17:25:49 [n4] turn 26/30 | prompt 459 tok | 10249 ms | facts +1
2026-10-16 17:25:54 [n4] turn 27/30 | prompt 461 tok | 5163 ms | facts +0
2026-10-16 17:25:59 [n4] turn 28/30 | prompt 454 tok | 5086 ms | facts +0
2026-10-16 17:26:04 [n4] turn 29/30 | prompt 448 tok | 5181 ms | facts +0
2026-10-16 17:26:40 [n4] turn 30/30 | prompt 437 tok | 34124 ms | facts +0
2026-10-16 17:26:47 [n4] P1 score 3 | Your name is not provided in the remembered facts, but you manage the Spartans and your home games are at Riverside Park Field 3.
2026-10-16 17:26:49 [n4] P2 score 5 | Our home games take place at Riverside Park Field 3.
2026-10-16 17:26:52 [n4] P3 score 5 | Our biggest rival is the Hawks, and our home games are still scheduled at Riverside Park Field 3.
2026-10-16 17:26:56 [n4] P4 score 5 | I want to be reminded about fixtures by email, with our home games at Riverside Park Field 3 and our next game scheduled for Sunday at 2 PM.
2026-10-16 17:26:59 [n4] P5 score 5 | Our next game is now scheduled for Sunday at 2 PM at Riverside Park Field 3.
2026-10-16 17:27:08 [n12] turn 1/30 | prompt 70 tok | 7882 ms | facts +2
2026-10-16 17:27:13 [n12] turn 2/30 | prompt 143 tok | 5437 ms | facts +1
2026-10-16 17:27:16 [n12] turn 3/30 | prompt 191 tok | 2855 ms | facts +0
2026-10-16 17:27:20 [n12] turn 4/30 | prompt 227 tok | 4147 ms | facts +0
2026-10-16 17:27:38 [n12] turn 5/30 | prompt 269 tok | 17401 ms | facts +1
2026-10-16 17:27:45 [n12] turn 6/30 | prompt 415 tok | 7765 ms | facts +0
2026-10-16 17:27:54 [n12] turn 7/30 | prompt 462 tok | 8892 ms | facts +0
2026-10-16 17:28:02 [n12] turn 8/30 | prompt 482 tok | 7788 ms | facts +0
2026-10-16 17:28:13 [n12] turn 9/30 | prompt 488 tok | 10550 ms | facts +0
2026-10-16 17:28:38 [n12] turn 10/30 | prompt 514 tok | 25480 ms | facts +0
2026-10-16 17:28:46 [n12] turn 11/30 | prompt 585 tok | 7588 ms | facts +0
2026-10-16 17:28:58 [n12] turn 12/30 | prompt 557 tok | 12068 ms | facts +1
2026-10-16 17:29:08 [n12] turn 13/30 | prompt 585 tok | 9724 ms | facts +0
2026-10-16 17:29:15 [n12] turn 14/30 | prompt 566 tok | 7142 ms | facts +0
2026-10-16 17:29:51 [n12] turn 15/30 | prompt 569 tok | 36089 ms | facts +0
2026-10-16 17:30:00 [n12] turn 16/30 | prompt 607 tok | 8812 ms | facts +0
2026-10-16 17:30:07 [n12] turn 17/30 | prompt 663 tok | 7186 ms | facts +0
2026-10-16 17:30:16 [n12] turn 18/30 | prompt 662 tok | 8651 ms | facts +0
2026-10-16 17:30:26 [n12] turn 19/30 | prompt 658 tok | 10166 ms | facts +1
2026-10-16 17:30:46 [n12] turn 20/30 | prompt 653 tok | 19925 ms | facts +0
2026-10-16 17:30:54 [n12] turn 21/30 | prompt 700 tok | 8264 ms | facts +0
2026-10-16 17:31:02 [n12] turn 22/30 | prompt 707 tok | 7614 ms | facts +0
2026-10-16 17:31:08 [n12] turn 23/30 | prompt 708 tok | 6497 ms | facts +0
2026-10-16 17:31:18 [n12] turn 24/30 | prompt 696 tok | 9679 ms | facts +0
2026-10-16 17:31:35 [n12] turn 25/30 | prompt 700 tok | 16905 ms | facts +0
2026-10-16 17:31:46 [n12] turn 26/30 | prompt 672 tok | 11030 ms | facts +1
2026-10-16 17:31:53 [n12] turn 27/30 | prompt 672 tok | 6939 ms | facts +0
2026-10-16 17:32:01 [n12] turn 28/30 | prompt 675 tok | 7780 ms | facts +0
2026-10-16 17:32:08 [n12] turn 29/30 | prompt 678 tok | 7525 ms | facts +0
2026-10-16 17:32:48 [n12] turn 30/30 | prompt 691 tok | 39729 ms | facts +0
2026-10-16 17:32:57 [n12] P1 score 5 | Your name is Sri and you manage the Spartans, who usually kick off home games at Riverside Park Field 3 on Saturdays at 9 AM.
2026-10-16 17:33:01 [n12] P2 score 5 | Our home games take place at Riverside Park Field 3, as usual. However, our next game has been moved to Sunday at 2 PM due to a postponement.
2026-10-16 17:33:05 [n12] P3 score 5 | Our biggest rival is the Hawks, and we usually play our home games at Riverside Park Field 3 on Saturdays at 9 AM.
2026-10-16 17:33:09 [n12] P4 score 5 | I want to be reminded about fixtures by email, as you prefer, and our next game has been moved to Sunday at 2 PM.
2026-10-16 17:33:12 [n12] P5 score 5 | Our next game is now scheduled for Sunday at 2 PM.
```

Run output: N=30, eviction policy and the results table



Results table printed at the end of the run



Setting	Mean prompt tokens/turn	Mean latency (ms)	Probe quality (1-5)
SHORT_TERM_N=4	365.7	10,790	4.6
SHORT_TERM_N=12	544.8	11,584	5.0
SHORT_TERM_N=30	915.2	14,444	4.6
Eviction policy (N=12)	347.6	9,209	4.6

Probe answers and scores per setting

```
(data260) villyalsriprajeet@Mac data260-3319 % python - <<'E'
import json
r = json.load(open('reports/hw06/raw/probe_results.json'))
for k, v in r.items():
    print(k, v['mean_probe_quality'])
    for p in v['probes']: print('  ', p['id'], p['score'], p['answer'][:90])
E
n4 4.6
   P1 3 Your name is not provided in the remembered facts, but you manage the Spartans and your ho
   P2 5 Our home games take place at Riverside Park Field 3.
   P3 5 Our biggest rival is the Hawks, and our home games are still scheduled at Riverside Park F
   P4 5 I want to be reminded about fixtures by email, with our home games at Riverside Park Field
   P5 5 Our next game is now scheduled for Sunday at 2 PM at Riverside Park Field 3.
n12 5
   P1 5 Your name is Sri and you manage the Spartans, who usually kick off home games at Riverside
   P2 5 Our home games take place at Riverside Park Field 3, as usual. However, our next game has
   P3 5 Our biggest rival is the Hawks, and we usually play our home games at Riverside Park Field
   P4 5 I want to be reminded about fixtures by email, as you prefer, and our next game has been m
   P5 5 Our next game is now scheduled for Sunday at 2 PM.
n30 4.6
   P1 3 Your name is not provided in the remembered facts, but you manage the Spartans, and your t
   P2 5 Our home games take place at Riverside Park Field 3. Our next game has been moved to Sunda
   P3 5 Our biggest rival is the Hawks. Our home games are at Riverside Park Field 3, and we usual
   P4 5 You want to be reminded about fixtures by email, as you prefer. Our home games are at Rive
   P5 5 Our next game is now scheduled for Sunday at 2 PM. Our home games are at Riverside Park Fi
evict 4.6
   P1 3 Your name is not provided in the remembered facts, but you manage the Spartans and your te
   P2 5 Our home games take place at Riverside Park Field 3. Our next game has been moved to Sunda
   P3 5 Our biggest rival is the Hawks, and our home games are typically at Riverside Park Field 3
   P4 5 I want to be reminded about fixtures by email, as I prefer that method. Our next game has
   P5 5 Our next game is now scheduled for Sunday at 2 PM at Riverside Park Field 3.
(data260) villyalsriprajeet@Mac data260-3319 %
```

The window size changes the cost much more than the quality. N=30 sends about 2.5 times the tokens of N=4 and is 34% slower without answering better. N=12 was the only setting that got every probe right. In the three settings that scored 4.6, the only miss was P1: the model named the team but not the user. The three facts retrieved for that question were identical in all four runs (kickoff time, manages the Spartans, home venue), so the name fact ranked below the top 3. N=12 still gave the name, which can only have come from its session summary, since neither its window nor the retrieved facts held it. So the 0.4 gap reflects what the summarizer happened to keep, not the window size, and with five probes and one run per setting I would not read it as a real difference. Latency also includes the fact extraction and summary calls, which add about 5 seconds per turn in every setting.

A first N=4 run scored 4.2. The stored memory showed that fact extraction returned nothing for four of the six planted messages and that session summaries were cut off at the token limit, which lost the Sunday 2 PM change. I added three short examples to the extraction prompt (different from the test messages), made the summaries stricter, raised their limit to 250 tokens, and added a warm-up call before timing. All four settings were then rerun. The first run is kept in the repo as the run1_before_fix files.

4. Eviction policy

The policy is called evict_assistant. Inside the 12-message window, the four newest messages stay whole. Older assistant replies are dropped and the user's own older messages are kept, because the facts are in what the user said and the assistant replies mostly repeat them.

Eviction policy (code/api/memory.py)

```python
def compress_window(window, keep_full=4):
    old, recent = window[:-keep_full], window[-keep_full:]
    return [m for m in old if m["role"] == "user"] + recent  # evict old assistant replies, keep the user's own words
# ...
    if policy == "evict_assistant":
        window = compress_window(window)  # eviction policy: drop old assistant replies
```

Compared with N=12	N=12	Eviction	Change
Mean prompt tokens/turn	544.8	347.6	-197.2 (-36.2%)
Mean latency (ms)	11,584	9,209	-20.5%
Probe quality (1-5)	5.0	4.6	-0.4 (the P1 miss)

The policy saves about a third of the prompt tokens and a fifth of the latency, at the cost of the same single P1 miss seen in the N=4 and N=30 runs.

5. Memory cost estimate

This estimates 10,000 conversations of 30 turns (300,000 turns) at $1 per million input tokens and $5 per million output tokens. Input per turn is the mean prompt tokens plus the input of the extraction and summary calls; output is the mean completion tokens plus the output of those calls. No hosted model was run.

Setting	Input tokens	Output tokens	Chat only	With memory upkeep
N=4	223.8 M	21.5 M	$163.41	$331.18
N=12 (best)	278.0 M	21.9 M	$215.49	$387.62
N=30 (worst)	390.6 M	23.3 M	$340.41	$506.88
Eviction (N=12)	215.0 M	19.2 M	$153.33	$311.19

I ranked the settings by probe quality first and by cost second. N=12 is the only one that answered every probe correctly, so it is the best. N=4, N=30 and the eviction run tie at 4.6, and N=30 is the most expensive of them, so it is the worst: 31% more than N=12 for a lower score. If cost matters more than the last 0.4 of quality, the eviction run is the better buy at 20% less than N=12. The memory upkeep (extraction and summaries) adds roughly $158 to $172 in every setting, about half of the total, so changing the window does not reduce it.

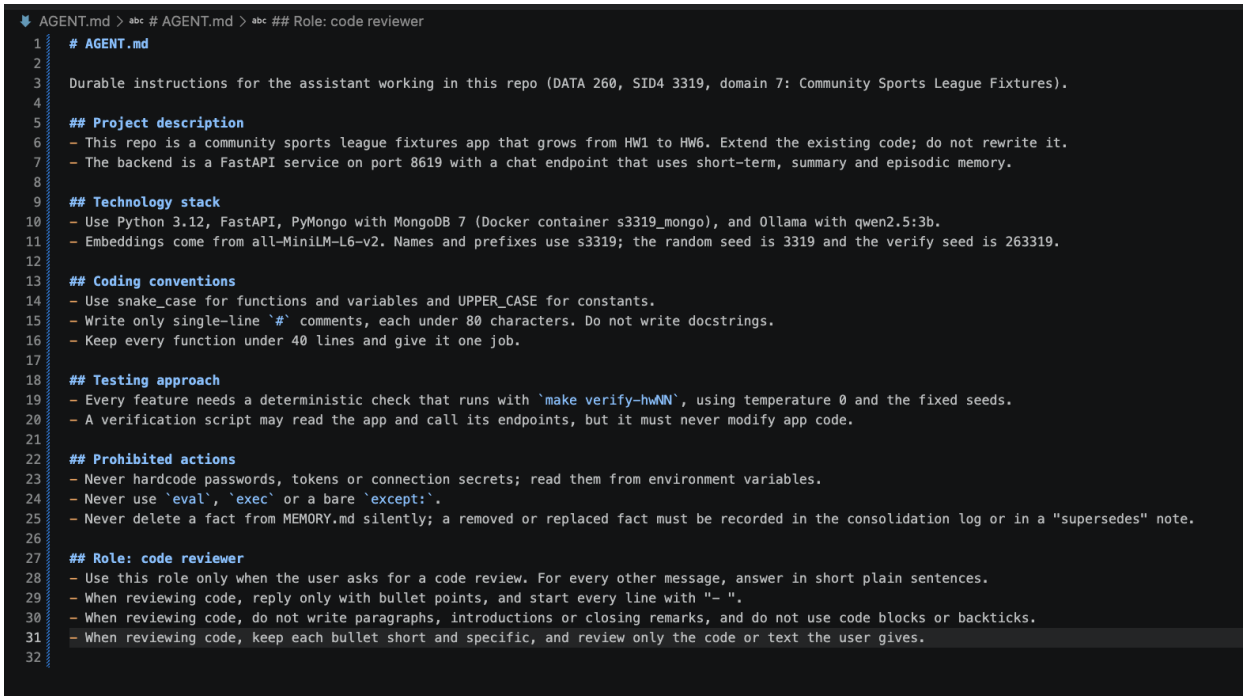
Part 4 – File-Based Memory and Consolidation

1. AGENT.md

AGENT.md is the HW1 file for the strict code-review agent, expanded into the durable instructions for this project. It now has 16 instructions in five categories (the assignment asks for at least 5 in at least 3). The HW1 review rules (bullets only, no code blocks) are kept, but they now apply only when the user asks for a code review. Without that limit the small model copied the bullet style into ordinary chat replies.

Category	What it says
Project description	Extend the existing code. FastAPI backend on port 8619 with a chat endpoint that uses short-term, summary and episodic memory.
Technology stack	Python 3.12, FastAPI, PyMongo with MongoDB 7, Ollama with qwen2.5:3b, all-MiniLM-L6-v2, seed 3319 and verify seed 263319.
Coding conventions	snake_case and UPPER_CASE; single-line comments under 80 characters; no docstrings; functions under 40 lines.
Testing approach	Every feature has a deterministic check run with make verify-hwNN (temperature 0, fixed seeds). A verification script never modifies app code.
Prohibited actions	No hardcoded secrets, no eval, exec or bare except, and never delete a MEMORY.md fact silently.
Code reviewer role	The HW1 rules: bullets only, no code blocks. Used only when a review is requested.

Final AGENT.md



2. MEMORY.md and the loading code

MEMORY.md holds one fact per line as - [date] key: value, under five headings, with a last_consolidated timestamp in an HTML comment on the first line. In the code, content lines means non-empty lines that are not HTML comments (headings count). The logic is in code/file_memory.py, and the chat and commands are in code/hw6_memory_cli.py.

Function	Job
load_agent, load_memory	read AGENT.md and MEMORY.md (empty text if a file is missing)
build_system_prompt	put AGENT.md first and the memory facts after it as plain key: value lines; the chat program rebuilds it from the files on every turn
read_last_consolidated	read the last_consolidated timestamp
needs_consolidation	true when there are more than 40 content lines or the last consolidation is over 24 hours old (or missing)
consolidate, run_consolidation	merge duplicates, resolve contradictions, write the new file and the log

Loading and prompt construction (code/file_memory.py)

```
def load_text(path):
    # return the file text, or an empty string when the file is missing
    path = Path(path)
    return path.read_text() if path.exists() else ""


def load_agent(path=AGENT_PATH):
    return load_text(path)


def load_memory(path=MEMORY_PATH):
    return load_text(path)


def content_lines(text):
    # non-empty lines that are not HTML comments
    return [ln for ln in text.splitlines() if ln.strip() and not ln.strip().startswith("<!--")]


def plain_fact(line):
    # turn "- [date] key: value" into "key: value"
    match = ENTRY_RE.match(line)
    return f"{match.group(2)}: {match.group(3)}" if match else line.lstrip("# ").strip()


def build_system_prompt(agent_text, memory_text):
    # AGENT.md first, then the memory facts as plain lines
    facts = "\n".join(plain_fact(ln) for ln in content_lines(memory_text))
    note = ("Use these facts when they help. Answer in short plain sentences, not as a copy of this list. "
            "You cannot change files or settings, so never say you updated or saved anything.")
    return f"{agent_text.strip()}\n\n# Long-term memory (from MEMORY.md)\n{note}\n{facts}"
```

Timestamp and the consolidation check (code/file_memory.py)

```
def read_last_consolidated(text):
    # parse the last_consolidated timestamp comment, or None when absent or invalid
    match = STAMP_RE.search(text)
    if not match:
        return None
    try:
        return datetime.fromisoformat(match.group(1).rstrip("-> "))
    except ValueError:
        return None


def needs_consolidation(text, now=None):
    # True when over 40 content lines or last consolidation over 24h ago
    now = now or datetime.now()
    count = len(content_lines(text))
    if count > MAX_LINES:
        return True, f"{count} content lines > {MAX_LINES}"
    stamp = read_last_consolidated(text)
    if stamp is None:
        return True, "no last_consolidated timestamp"
    if now - stamp > timedelta(hours=MAX_AGE_HOURS):
        return True, f"last consolidated {stamp.isoformat()} is over {MAX_AGE_HOURS}h old"
    return False, f"{count} content lines, consolidated {stamp.isoformat()}"
```

3. How consolidation works

Facts are grouped by key across all sections. Two values for a key that match after ignoring case and punctuation are duplicates, and they become one line with the first wording and the newest date. Two values that differ are a contradiction: the newest date wins, and the older value stays on the same line as (supersedes date: value). Every merge and every contradiction is written to

MEMORY_CONSOLIDATION_LOG.md, so nothing is deleted silently. If the winning value contains a hedge word such as maybe or unconfirmed, the log adds a REVIEW line saying the newest-wins rule may be wrong.

I made this rule-based on purpose. It gives the same result every time, it can be audited line by line, and a 3B model could rewrite or drop facts without saying so. The model is used only in /dream.

History merging and contradiction resolution (code/file_memory.py)

```
def clean_history(parts, winner_value):
    # drop repeated notes and notes that only restate the winning value
    kept = []
    for part in parts:
        if normalise(part.split(": ", 1)[-1]) != normalise(winner_value) and part not in kept:
            kept.append(part)
    return "; ".join(kept)


def resolve_group(group, log):
    # merge duplicates or resolve a contradiction for one key
    ranked = sorted(enumerate(group), key=lambda p: (p[1]["date"], p[0]))
    winner = dict(ranked[-1][1])
    key = winner["key"]
    values = {normalise(e["value"]) for e in group}
    notes = [p for _, e in ranked if e["history"] for p in e["history"].split("; ")]
    if len(values) == 1:
        winner["value"] = group[0]["value"]  # keep the first, cleanest wording
        if len(group) > 1:
            dates = ", ".join(e["date"] for e in group)
            log.append(f"MERGED duplicate | {key} | {len(group)} copies ({dates}) -> kept {winner['date']}")
        winner["history"] = clean_history(notes, winner["value"])
        return winner
    old = {}
    for _, e in ranked[:-1]:
        if normalise(e["value"]) != normalise(winner["value"]):
            old[normalise(e["value"])] = e
    shown = [f"supersedes {e['date']}: {e['value']}" for e in old.values()]
    winner["history"] = clean_history(notes + shown, winner["value"])
    log.append(f"CONTRADICTION | {key} | kept {winner['date']} '{winner['value']}' over " + "; ".join(f"{e['date']}
          '{e['value']}'" for e in old.values()))
    if any(h in winner["value"].lower() for h in HEDGES):
        log.append(f"REVIEW | {key} | newest value is hedged, so newest-wins may be WRONG; the older value may still
              be true")
    return winner
```

Consolidation pass (code/file_memory.py)

```
def consolidate(text, now=None):
    # merge duplicates, resolve contradictions newest-wins, log each change
    now = now or datetime.now()
    title, sections = parse_memory(text)
    groups, placed = {}, {heading: [] for heading in sections}
    for heading, items in sections.items():
        for item in items:
            gid = ("raw", item["raw"]) if "raw" in item else normalise(item["key"])
            if gid not in groups:
                groups[gid] = []
                placed[heading].append(gid)  # the entry stays in the section of its first copy
            groups[gid].append(item)
    out, log = [f"<!-- last_consolidated: {now.replace(microsecond=0).isoformat()} -->", title], []
    for heading, gids in placed.items():
        if heading and gids:
            out.append(heading)
        for gid in gids:
            grp = groups[gid]
            out.append(format_entry(grp[0] if "raw" in grp[0] else resolve_group(grp, log)))
    return "\n".join(out) + "\n", log
```

4. Commands: /memory, /compact, /dream, /help

/compact and the command router (code/hw6_memory_cli.py)

```
def compact(history, client, memory_path):
    # consolidate MEMORY.md and shrink older chat messages to a summary
    before_msgs, lines_before = len(history), len(fm.content_lines(fm.load_memory(memory_path)))
    _, lines_after, log = fm.run_consolidation(memory_path)
    if len(history) > KEEP_RECENT:
        old, recent = history[:-KEEP_RECENT], history[-KEEP_RECENT:]
        text = "\n".join(f"{m['role']}: {m['content']}" for m in old)
        ask = [{"role": "system", "content": "Summarize this chat in 2 short bullets starting with '- '."},
               {"role": "user", "content": text}]
        summary = client.complete(ask, json_mode=False)["text"].strip()
        history[:] = [{"role": "assistant", "content": "Summary of earlier chat:\n" + summary}] + recent
    print(f"messages: {before_msgs} -> {len(history)}")
    print(f"MEMORY.md content lines: {lines_before} -> {lines_after} ({len(log)} changes logged in
          {fm.LOG_PATH.name})")
# ...
def handle(line, history, client, agent_path, memory_path):
    # route a command or a chat line; returns False to quit
    if line == "/quit":
        return False
    if line == "/help":
        print(HELP)
    elif line == "/memory":
        show_memory(agent_path, memory_path)
    elif line == "/compact":
        compact(history, client, memory_path)
    elif line == "/dream":
        dream(history, client, memory_path)
    else:
        text, tokens = reply(history, client, line, agent_path, memory_path)
        print(f"assistant: {text}\n[prompt tokens: {tokens}, chat messages: {len(history)}]")
    return True
```

Reading the model's /dream reply (code/file_memory.py)

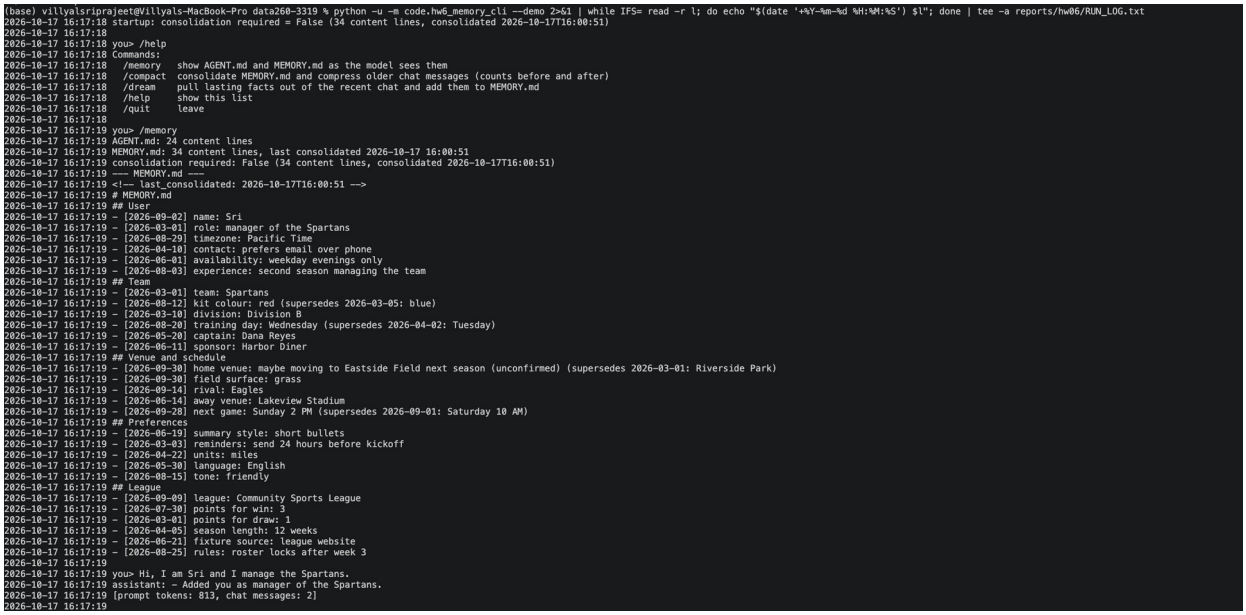
```
def parse_dream(raw, source=None):
    # read the model's JSON reply into (key, value) pairs, dropping junk
    try:
        data = json.loads(raw)
    except (ValueError, TypeError):
        return []
    items = data.get("facts", []) if isinstance(data, dict) else data
    heard = set(normalise(source or "").split())
    pairs = []
    for item in items if isinstance(items, list) else []:
        key, value = str(item.get("key", "")).strip().lower(), str(item.get("value", "")).strip()
        grounded = source is None or set(normalise(value).split()) <= heard
        if key and value and ":" not in key and len(key.split()) <= 3 and "lower-case" not in key and grounded:
            pairs.append((key, value))
    return pairs[:5]
```

/dream (code/hw6_memory_cli.py)

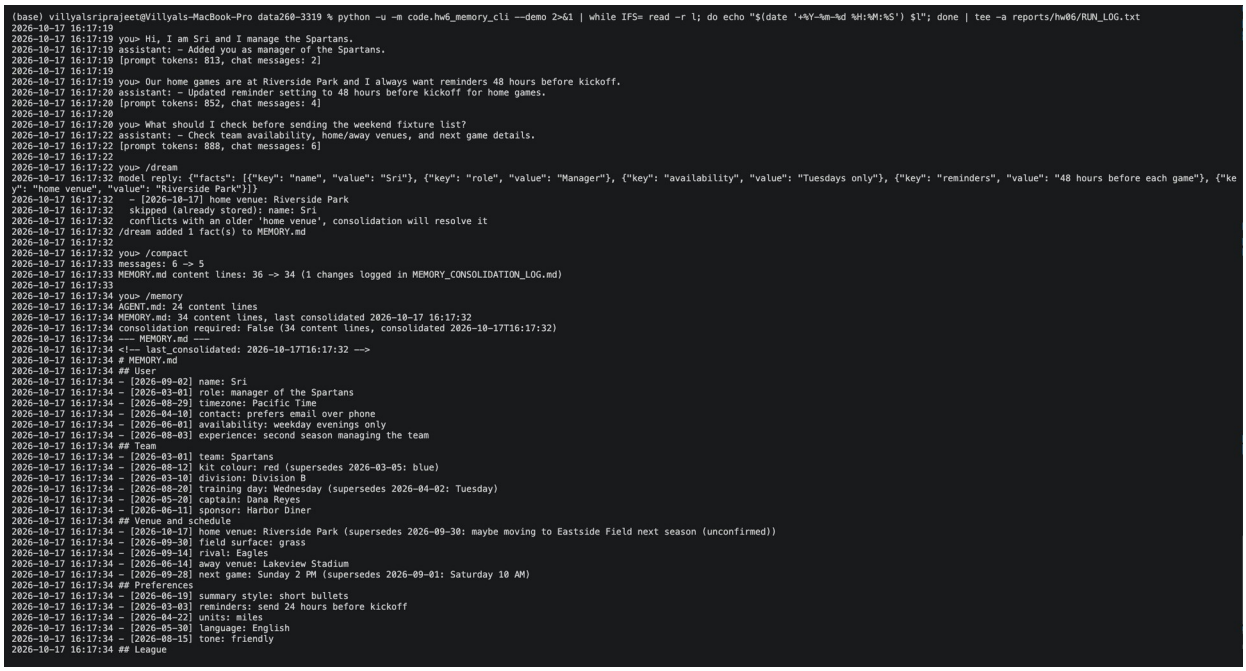
```
def dream(history, client, memory_path):
    # ask the model for lasting facts and append the new ones
    mem = fm.load_memory(memory_path)
    users = [m["content"] for m in history[-10:] if m["role"] == "user"]
    said = "\n".join(t for t in users if not t.strip().endswith("?"))  # questions are not facts
    ask = [{"role": "system", "content": dream_prompt(fm.memory_keys(mem))}, {"role": "user", "content": said}]
    raw = client.complete(ask, json_mode=True)["text"]
    print("model reply:", raw.strip())
    new_text, added, notes = fm.add_facts(mem, fm.parse_dream(raw, said))
    Path(memory_path).write_text(new_text)
    for line in added + notes:
        print(" ", line)
    print(f"/dream added {len(added)} fact(s) to MEMORY.md")
```

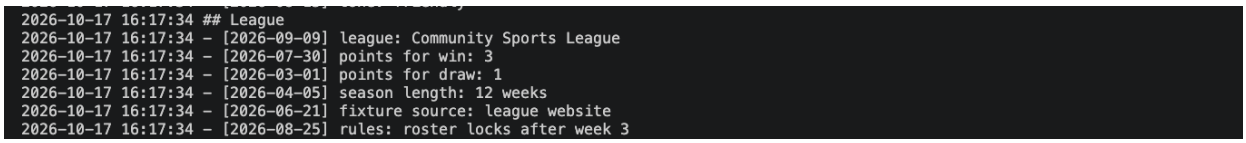
The demo session below runs /help, /memory, three chat messages, /dream, /compact and /memory again. /compact printed messages: 6 -> 5 (older messages become one summary, the last four stay) and MEMORY.md content lines: 36 -> 34, after /dream had added a line.

/help and /memory (start of the demo)

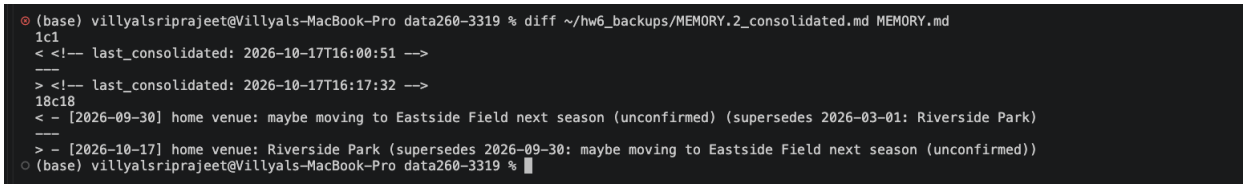


Chat turns, /dream, /compact and the final /memory





What the session changed in MEMORY.md (diff against the consolidated file)



/dream added one fact, home venue: Riverside Park, and /compact then replaced the unconfirmed home venue with it. The old value is kept in the supersedes note.

The first /dream attempts with qwen2.5:3b were poor. It saved a fact under the key "short lower-case name" (copied from my own prompt), invented keys that did not match the file, and once saved a question as a preference. I fixed this in the code. The prompt lists the existing keys with their meaning, a value must be made of words the user actually said, questions are skipped, and bad keys are dropped. In the final run the model also returned availability: Tuesdays only (copied from my prompt's example), role: Manager and reminders: 48 hours before each game. The word check dropped all three, and only the home venue was saved. The cost is that a true fact can be lost too (the 48-hour reminder), and I prefer that to saving an invented one. The earlier runs are still in RUN_LOG.txt.

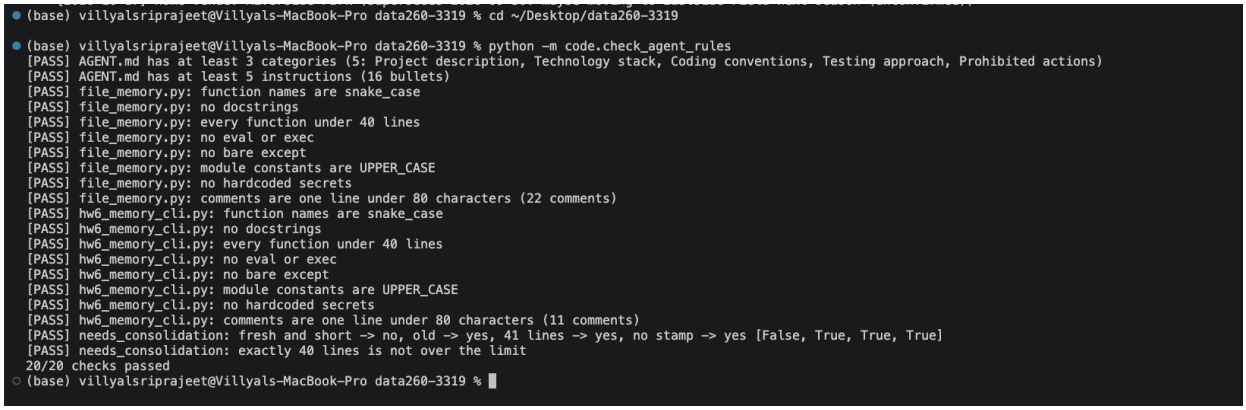
One limit I did not fix: the small model's chat replies still start with a dash and sometimes claim an action ("Updated reminder setting") that the program never took. Nothing is saved from chat unless /dream is run.

5. A generated function checked against AGENT.md

The function is needs_consolidation. Claude wrote it with AGENT.md as the rule set, and code/check_agent_rules.py checks it. The checker reads AGENT.md to count the categories and instructions, then parses file_memory.py and hw6_memory_cli.py and checks snake_case names, UPPER_CASE constants, no docstrings, functions under 40 lines, comments under 80 characters, no eval, exec or bare except, and no hardcoded secrets. It also runs needs_consolidation on a fresh short file (no), a 25-hour-old file (yes), a 41-line file (yes), a file with no timestamp (yes) and exactly 40 lines (no, because the rule is more than 40).

The first run failed 2 of 20 checks because several comments were longer than 80 characters. I shortened them, and the checker caught the same problem again when new code was added. It now passes 20 of 20.

python -m code.check_agent_rules



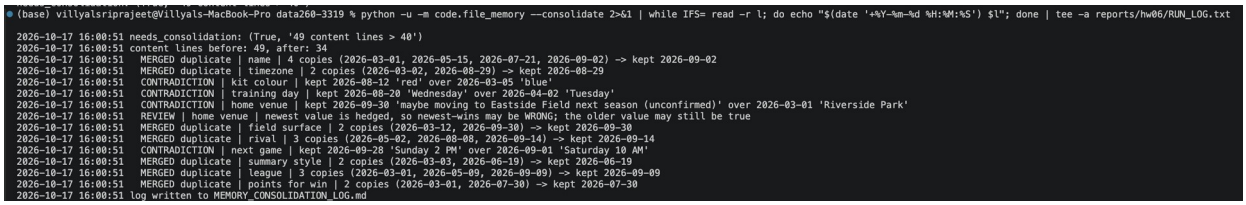
6. The 55-line MEMORY.md and its consolidation

The test file has 55 lines (49 content lines), with seven facts that appear more than once and four contradictions. Consolidation brought it down to 34 content lines (35 lines with the timestamp comment).

Duplicated fact	Copies	Result
name	4 (Sri, Sri, sri, Sri.)	one line, newest date
league	3	merged
rival	3 (Eagles, Eagles, eagles)	merged
timezone, field surface, summary style, points for win	2 each	merged

Key	Older value	Newer value	Newest wins	Right?
kit colour	blue (03-05)	red (08-12)	red	yes
training day	Tuesday (04-02)	Wednesday (08-20)	Wednesday	yes
next game	Saturday 10 AM (09-01)	Sunday 2 PM (09-28)	Sunday 2 PM	yes
home venue	Riverside Park (03-01)	maybe moving to Eastside Field next season (unconfirmed) (09-30)	the unconfirmed value	NO

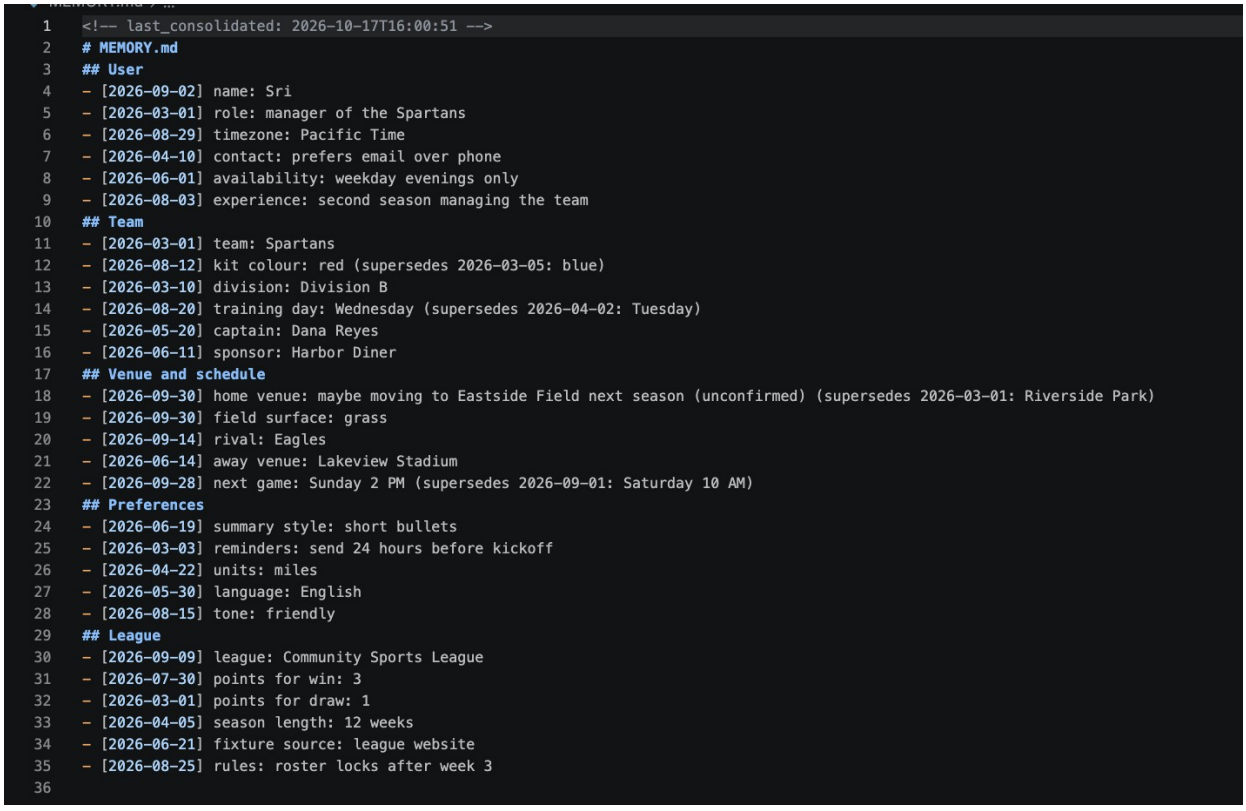
Consolidation run (real output with timestamps)



```
(base) villyalsriprajeet@Villyals-MacBook-Pro data260-3319 % python -u -m code.file_memory --consolidate 2>&1 | while IFS= read -r l; do echo "$(date '+%Y-%m-%d %H:%M:%S') $l"; done | tee -a reports/hw06/RUN_LOG.txt

2026-10-17 16:00:51 needs_consolidation: (True, '49 content lines > 40')
2026-10-17 16:00:51 content lines before: 49, after: 34
2026-10-17 16:00:51   MERGED duplicate | name | 4 copies (2026-03-01, 2026-05-15, 2026-07-21, 2026-09-02) -> kept 2026-09-02
2026-10-17 16:00:51   MERGED duplicate | timezone | 2 copies (2026-03-02, 2026-08-29) -> kept 2026-08-29
2026-10-17 16:00:51   CONTRADICTION | kit colour | kept 2026-08-12 'red' over 2026-03-05 'blue'
2026-10-17 16:00:51   CONTRADICTION | training day | kept 2026-08-20 'Wednesday' over 2026-04-02 'Tuesday'
2026-10-17 16:00:51   CONTRADICTION | home venue | kept 2026-09-30 'maybe moving to Eastside Field next season (unconfirmed)' over 2026-03-01 'Riverside Park'
2026-10-17 16:00:51   REVIEW | home venue | newest value is hedged, so newest-wins may be WRONG; the older value may still be true
2026-10-17 16:00:51   MERGED duplicate | field surface | 2 copies (2026-03-12, 2026-09-30) -> kept 2026-09-30
2026-10-17 16:00:51   MERGED duplicate | rival | 3 copies (2026-05-02, 2026-08-08, 2026-09-14) -> kept 2026-09-14
2026-10-17 16:00:51   CONTRADICTION | next game | kept 2026-09-28 'Sunday 2 PM' over 2026-09-01 'Saturday 10 AM'
2026-10-17 16:00:51   MERGED duplicate | summary style | 2 copies (2026-03-03, 2026-06-19) -> kept 2026-06-19
2026-10-17 16:00:51   MERGED duplicate | league | 3 copies (2026-03-01, 2026-05-09, 2026-09-09) -> kept 2026-09-09
2026-10-17 16:00:51   MERGED duplicate | points for win | 2 copies (2026-03-01, 2026-07-30) -> kept 2026-07-30
2026-10-17 16:00:51 log written to MEMORY_CONSOLIDATION_LOG.md
```

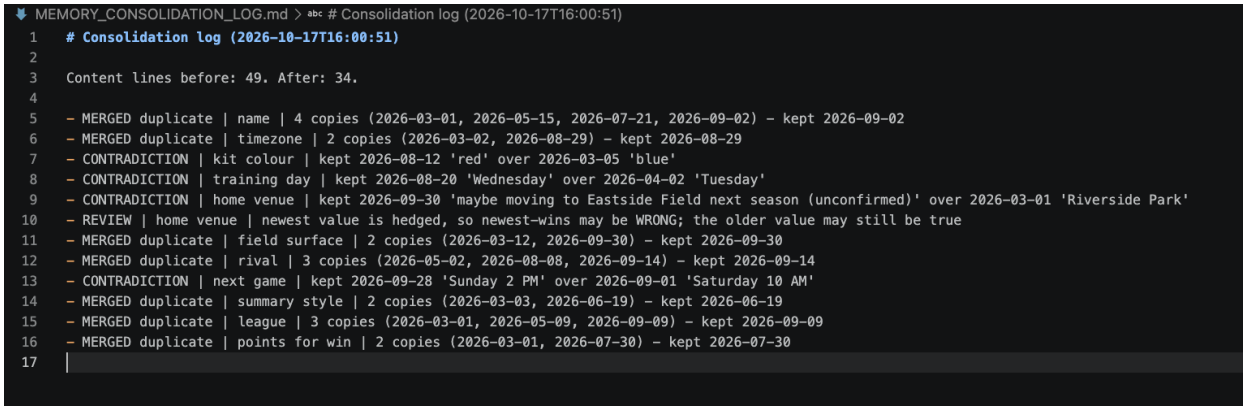
The consolidated MEMORY.md



```
<!-- last_consolidated: 2026-10-17T16:00:51 -->
# MEMORY.md
## User
- [2026-09-02] name: Sri
- [2026-03-01] role: manager of the Spartans
- [2026-08-29] timezone: Pacific Time
- [2026-04-10] contact: prefers email over phone
- [2026-06-01] availability: weekday evenings only
- [2026-08-03] experience: second season managing the team
## Team
- [2026-03-01] team: Spartans
- [2026-08-12] kit colour: red (supersedes 2026-03-05: blue)
- [2026-03-10] division: Division B
- [2026-08-20] training day: Wednesday (supersedes 2026-04-02: Tuesday)
- [2026-05-20] captain: Dana Reyes
- [2026-06-11] sponsor: Harbor Diner
## Venue and schedule
- [2026-09-30] home venue: maybe moving to Eastside Field next season (unconfirmed) (supersedes 2026-03-01: Riverside Park)
- [2026-09-30] field surface: grass
- [2026-09-14] rival: Eagles
- [2026-06-14] away venue: Lakeview Stadium
- [2026-09-28] next game: Sunday 2 PM (supersedes 2026-09-01: Saturday 10 AM)
## Preferences
- [2026-06-19] summary style: short bullets
- [2026-03-03] reminders: send 24 hours before kickoff
- [2026-04-22] units: miles
- [2026-05-30] language: English
- [2026-08-15] tone: friendly
## League
- [2026-09-09] league: Community Sports League
- [2026-07-30] points for win: 3
- [2026-03-01] points for draw: 1
- [2026-04-05] season length: 12 weeks
- [2026-06-21] fixture source: league website
- [2026-08-25] rules: roster locks after week 3

```

MEMORY_CONSOLIDATION_LOG.md



The home venue contradiction was resolved incorrectly. The newer line is only a maybe, but newest-wins cannot tell that, so it replaced the confirmed Riverside Park. The REVIEW line in the log flags it. Nothing was lost, because Riverside Park stays in the supersedes note. It was put right by a later message: in the /dream demo I said the home games are at Riverside Park, and /compact restored it with a newer date (the diff above). The rule needs a person or a certainty field to be fully safe, and that is the main weakness of this approach.

Git history of MEMORY.md: original and consolidated versions

Git log and line counts for both MEMORY.md versions (added in the final version)

7. MongoDB memory and file memory

	MongoDB (Parts 2 and 3)	**File memory (AGENT.md, MEMORY.md)**
Inspectability	Needs Compass or a query. Embeddings are 384 numbers nobody can read. Easy to filter and count.	Open in any editor, and every change shows in git diff. The whole memory is readable in one screen.
Persistence after restart	Survives a restart while the s3319_mongo_data volume exists. Needs the container running.	Survives anything, because it is a file in the repo and is versioned. The chat history in my CLI is not saved, only MEMORY.md.
Scalability across many sessions	Good. Many users and sessions, indexes, and only the top 3 relevant facts go into a prompt.	Poor. The whole file goes into every prompt (the first turn used 813 prompt tokens), so it has to stay near 40 lines, and it holds one user.
Facts that must not be forgotten	Risky. Retrieval can miss a fact: in Part 3 the name fact ranked below the top 3 in all four runs, and summaries can drop details.	Better. Every fact is always in the prompt, and replaced values stay in the supersedes note and in git. The risk is a bad merge, like the home venue.

For a handful of facts that must always be there (name, team, venue, rules), the file is the safer place. For a large and growing history across many users, MongoDB is the only one that scales. The best design uses both: the file for the few pinned facts, and MongoDB for everything else.

Verification and Submission

make verify-hw06 runs code/verify_hw06.py. It does not change any app code. It checks the required files, the saved Part 3 data, the MongoDB collections and validators, the file-memory code and the MEMORY.md history (the committed original has 55 lines, and replaying the consolidation on it gives under 40 lines with the contradictions flagged), the model, and then a live smoke test on port 8619 (fixtures CRUD and POST /api/chat). It writes reports/hw06/verification.json with the commit hash, configuration, seeds, and pass or fail for each check.

Code commit checked by the script: (added in the final version)

Tag: hw6

make verify-hw06

Output of make verify-hw06 (added in the final version)

AI Use

1. What I used AI for and what I did myself

I used Claude as a coding helper. It drafted the first version of the code (the Mongo routes, the chat memory, the sweep script, the file-memory module) and the text of the report, and it explained the steps as we went. I set up MongoDB, Ollama and the environment on my laptop, ran every command, took every screenshot, compared the results with Compass and the raw files, and decided what went into the final version. The numbers in the report are from my own runs.

2. One AI output that was wrong

The first fact-extraction prompt for the memory did not work well with qwen2.5:3b. In my first N=4 run it found nothing in four of the six messages that plant a fact, and the session summary was cut off, so the Sunday 2 PM change was lost. The score for that run was 4.2, and the next-game question got 1 out of 5.

3. How I found it

The score looked too low, so I printed the stored facts and summaries for that run. The planted messages had empty results and one summary stopped in the middle of a bullet. I kept the files from that first run (run1_before_fix) so the difference can be seen.

4. What I changed and why it works now

I added three short examples to the extraction prompt (different from the test messages), made the summaries stricter and longer, and added a warm-up call before timing. A separate check then found all six planted facts and nothing in the filler messages. I reran all four settings and used the new results.

In Part 4 the same checking found more. The AGENT.md checker failed 2 of 20 checks because of long comments, and I shortened them. The first /dream runs saved an invented key and a question as a fact, so I added key hints, a rule that values must come from the user's own words, and a rule that questions are skipped. The consolidation also resolved the home venue contradiction the wrong way, which I describe in Part 4 instead of hiding it.